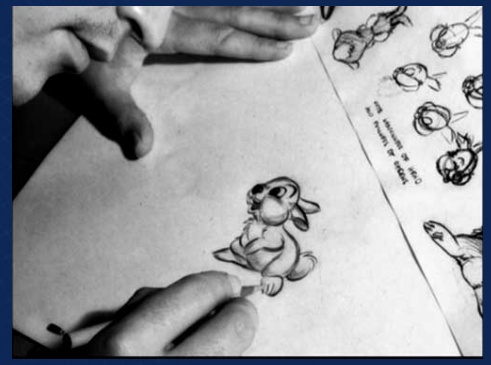
Vorlesung: Grundlagen der Computergrafik

Kapitel 6:

Computer Animation



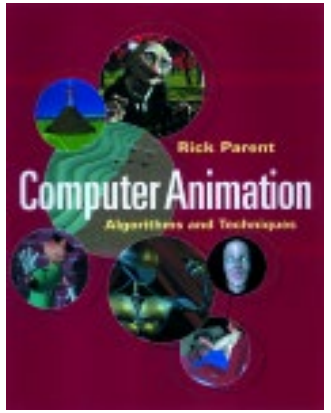


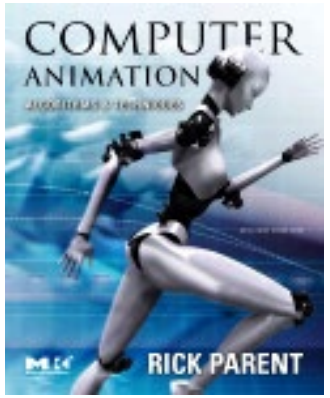
Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier

Weiter zu

- Kapitel 6.1: Einleitung
- Kapitel 6.2: Keyframe Animation
- Kapitel 6.3: Andere Animationstechniken

Weiterführende Literatur:
Rick Parent,
Computer Animation, Algorithms and Techniques,
Morgan Kaufmann, 2002, 2012.





Vorlesung: Grundlagen der Computergrafik

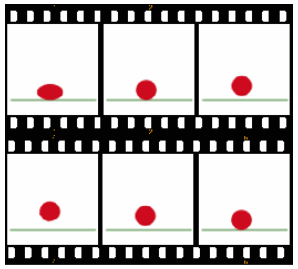
Kapitel 6: Computer Animation



Kapitel 6.1:
Introduction

Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier



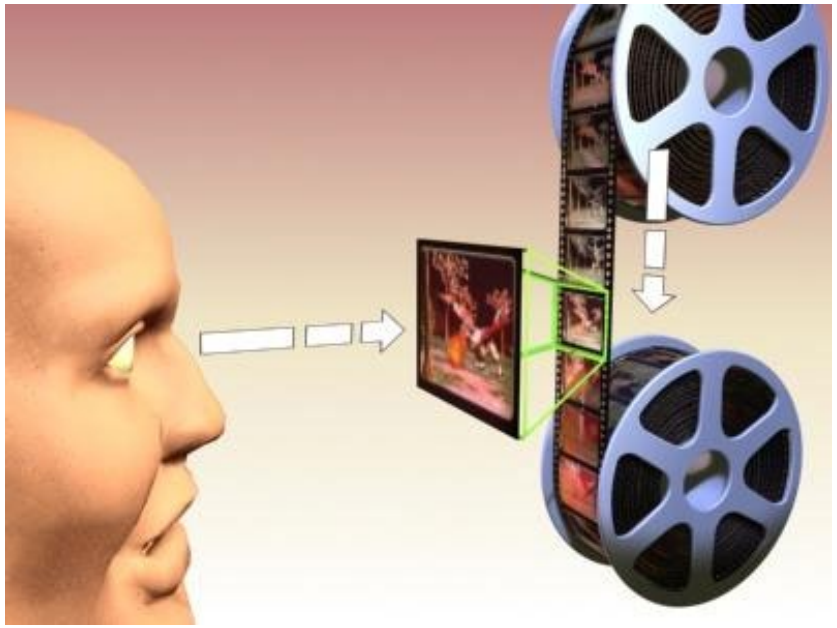


Was ist Animation?



- Folge von Bildern eines Objekts/Figur („Charakter“) in Bewegung, die mit einer bestimmten Geschwindigkeit nacheinander angezeigt werden:

- typische Frameraten (>14-16 fps):
  - Film 24 fps (1440 Bilder pro Minute)
  - Video 30 fps (1800 Bilder pro Minute)



Beispiele

- Daumenkino
- klassische Animation von Hand
- Stop Motion („claymation“, Knetanimation), z.B. Wallace&Gromit, diverse Lego-Animationen
- Computererzeugte Animation aus Schlüsselbildern (computer-assisted keyframing)
- Motion Capture, z.B. Happy Feet, Avatar
- (physikalische) Simulation





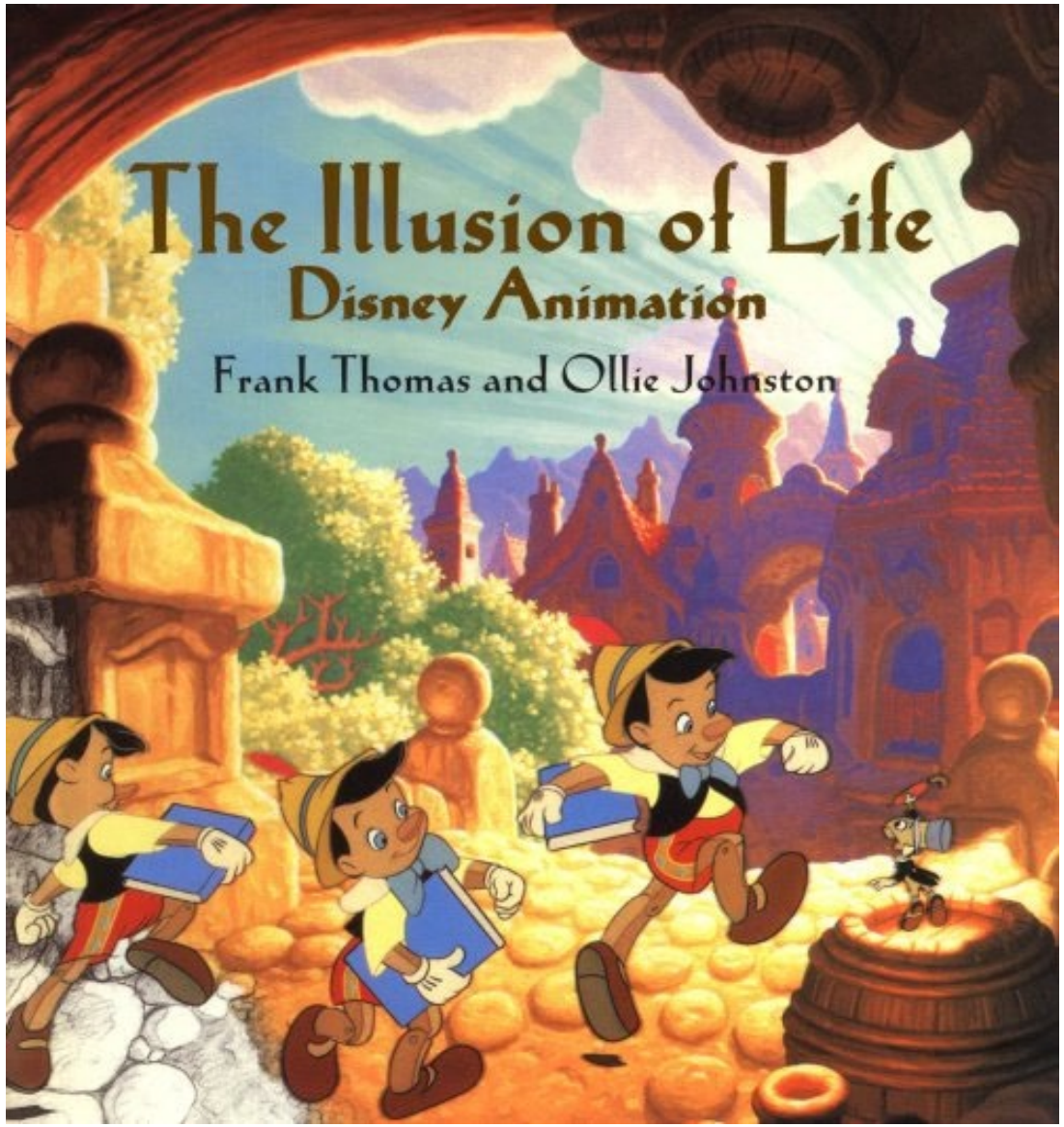


Exkurs

Überspringen

„Klassische“ Animationsprinzipien

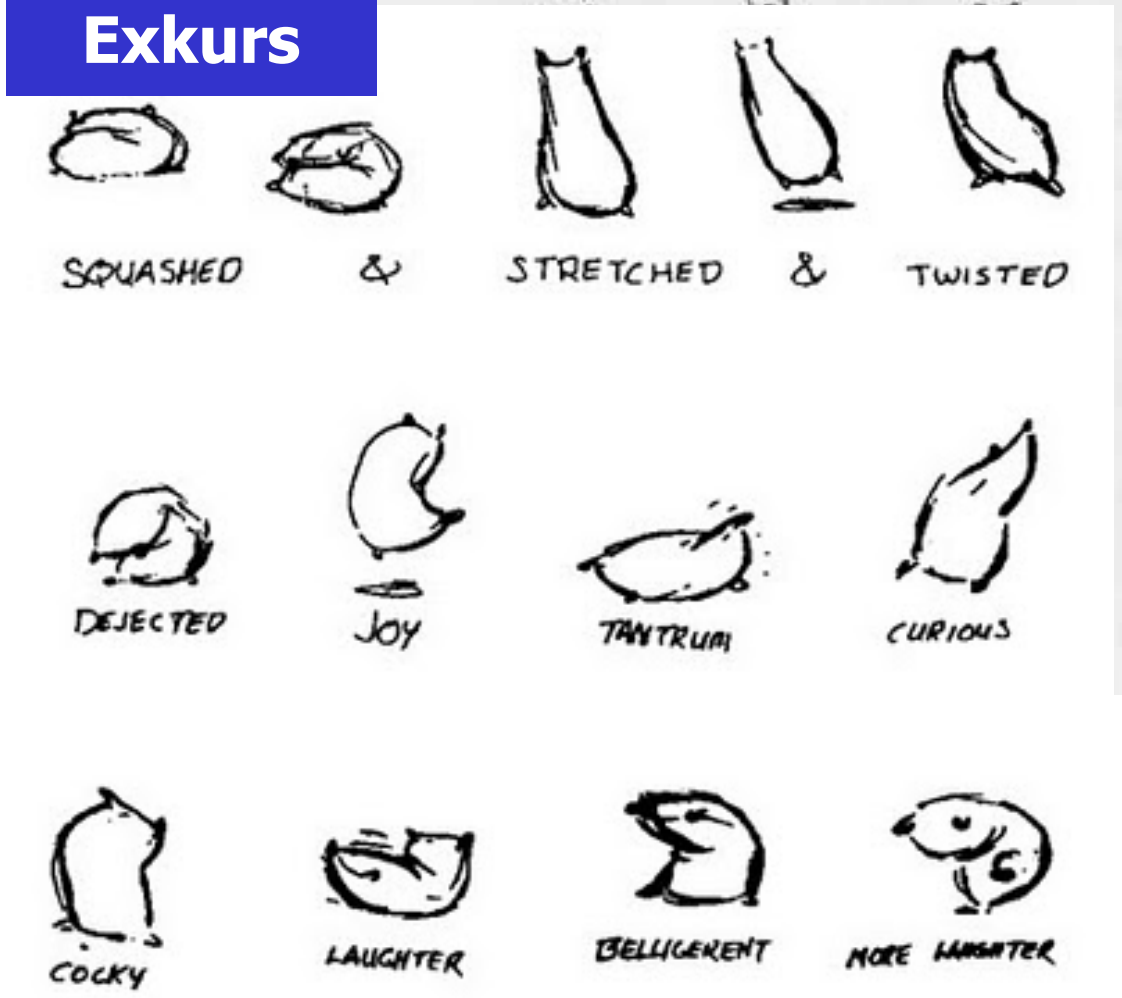
- aus „The Illusion of Life“ von Frank Thomas, Ollie Johnston, 1981

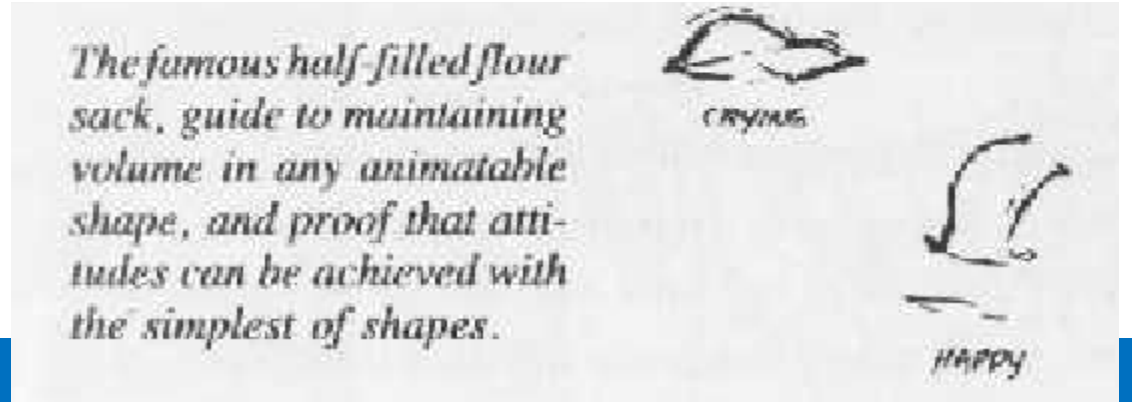


„Klassische“ Animationsprinzipien

- Squash and stretch
- Anticipation
- Staging
- Pose to Pose
- Follow Through
- Slow In, Slow Out
- Arcs
- Secondary Action
- Timing
- Exaggeration
- Solid Drawing
- Appeal

Exkurs





„Klassische" Animations-prinzipien

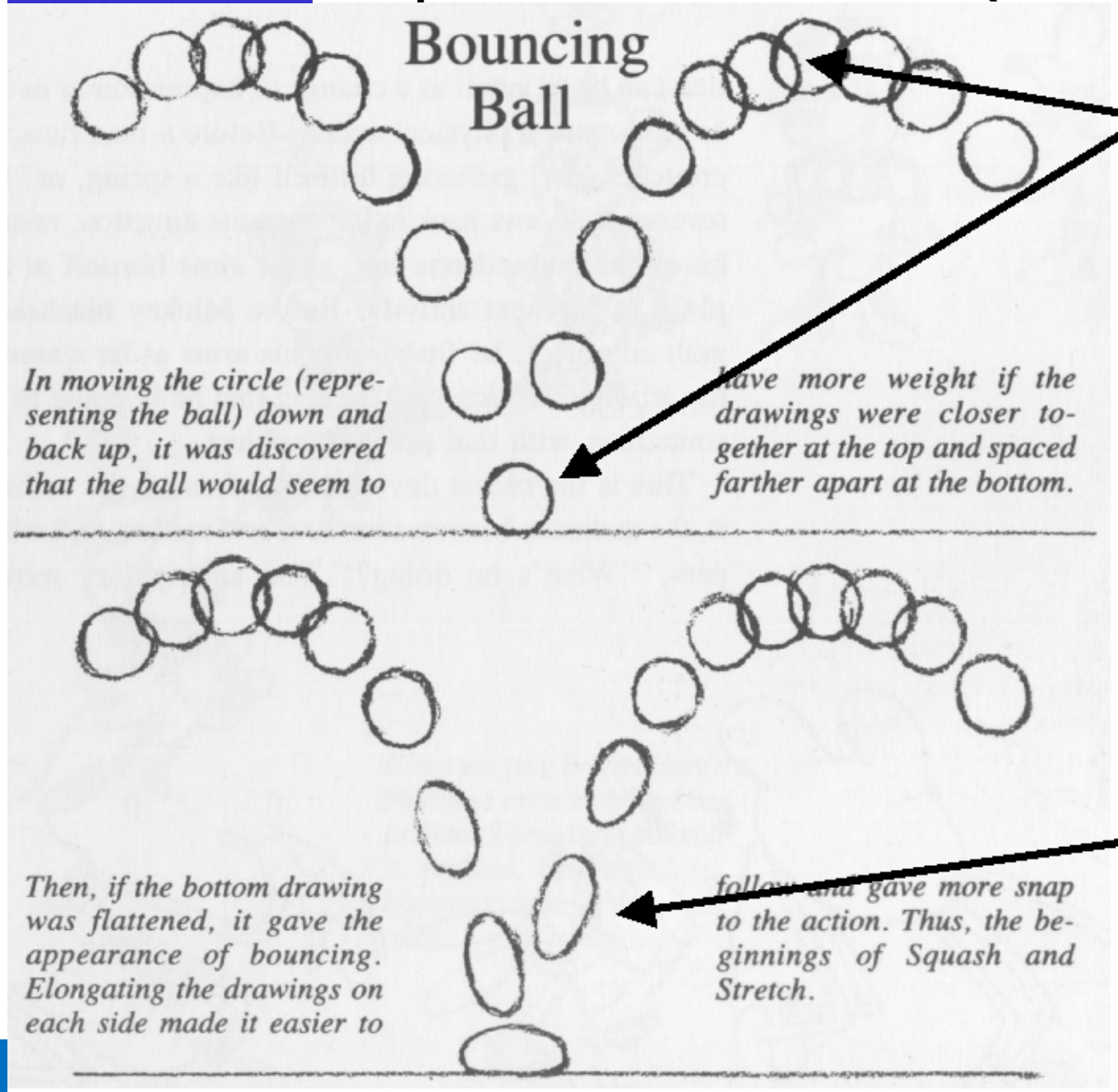
Die Disney Animateure demonstrieren die Prinzipien, mit einem halbgefüllten Mehlsack.

Figuren bestehen aus Fleisch und Blut, sind lebendig und bewegen sich nicht steif, da sie Haut, Fleisch und Knochen bestehen.

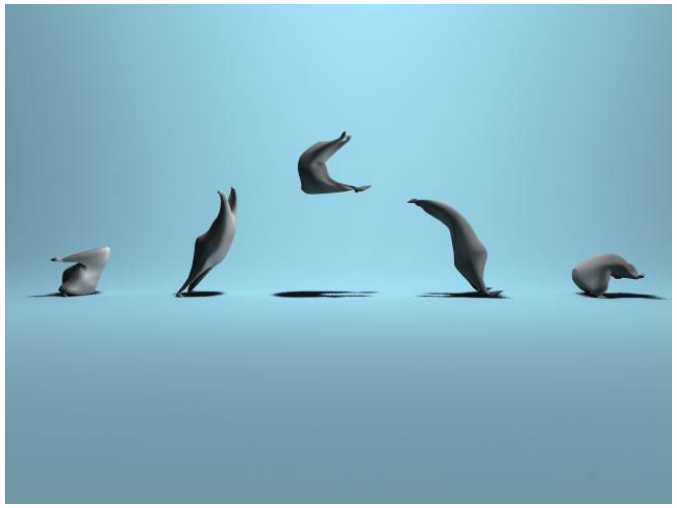
Das Volumen muss bei Bewegungen erhalten bleiben.

Exkurs

Squash and Stretch (Stauchen&Strecken)

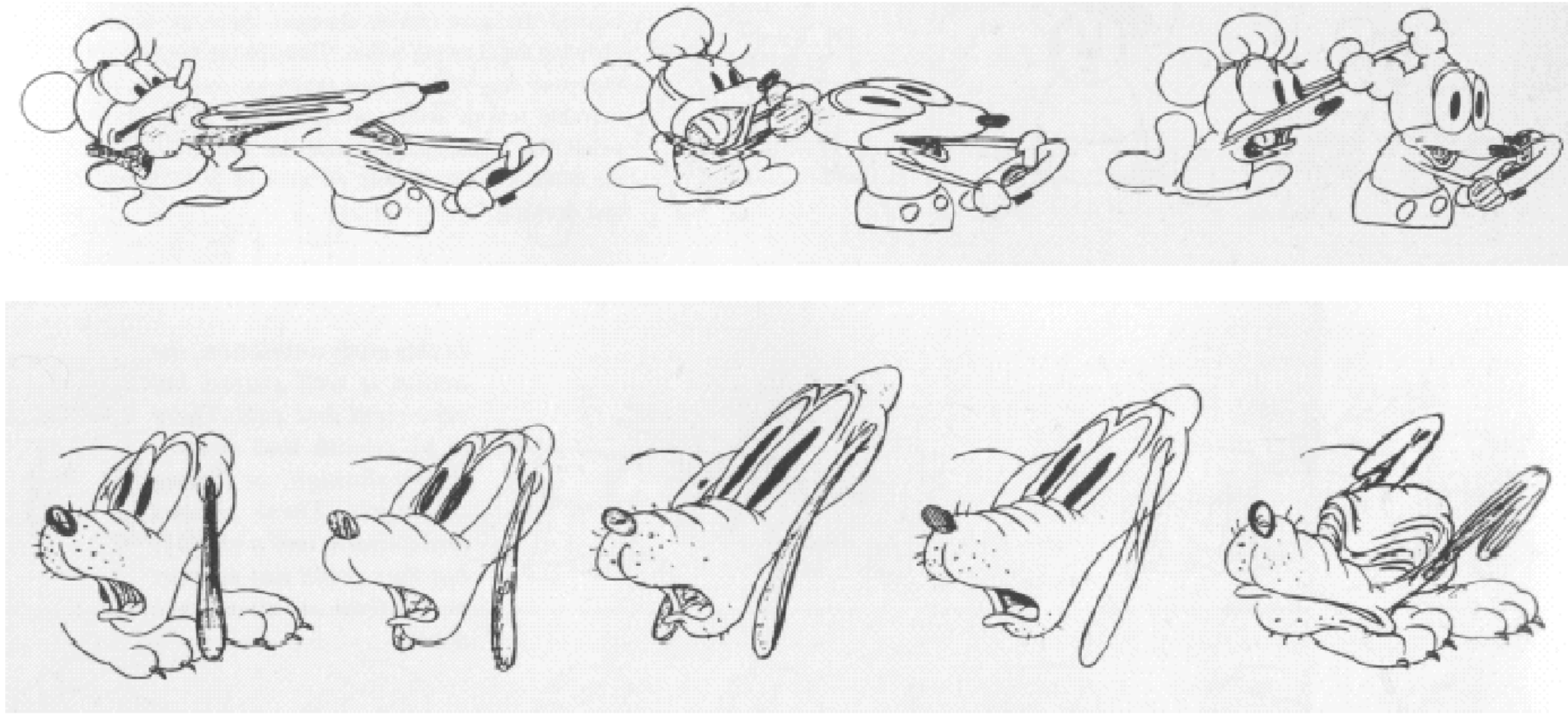


Durch das Timing entsteht der Eindruck, der Kreis (Ball) habe Gewicht.



Hinzufügen von Squash & Stretch macht Bewegung + Geschwindigkeit des Balles deutlicher. (eine Art 2D Motion Blur, Bewegungsunschärfe)

Exkurs Squash and Stretch (Stauchen&Strecken)



In den Beispielen wird das Volumen annährend erhalten, aber die Form drastisch verändert.

Exkurs

Anticipation and Staging

(Erwartung erwecken & in Phasen zerlegen)

Vorbereiten einer Action ⟶ Ausführen einer Action

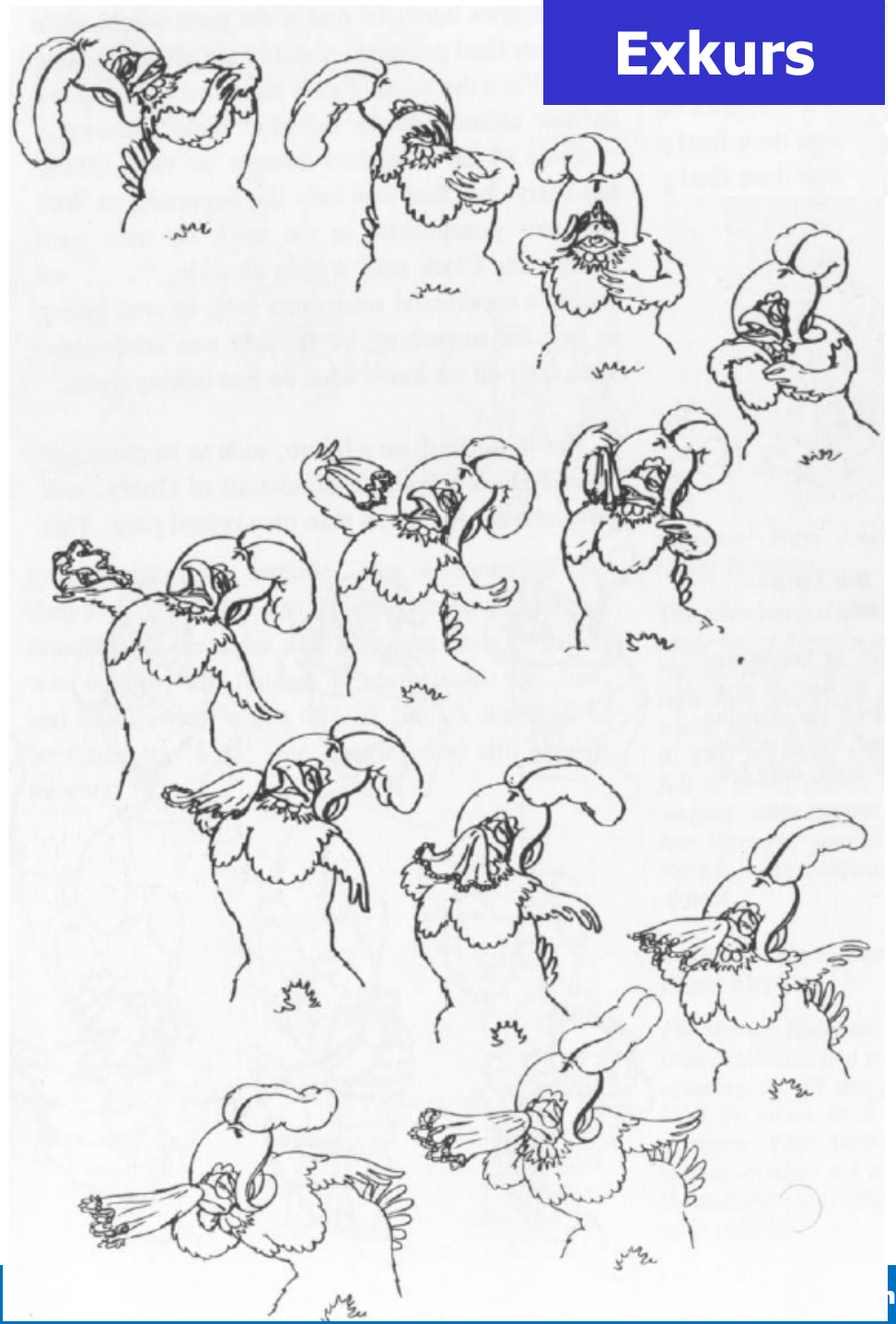


Lass die Zuschauer wissen, was du vorhast, bevor du es wirklich tust, z.B. durch einen Gesichtausdruck oder durch Körperverdrehung.

Überrasche deine Zuschauer nur mit Absicht, nicht ungewollt!

Was hat Donald vor ?

Exkurs



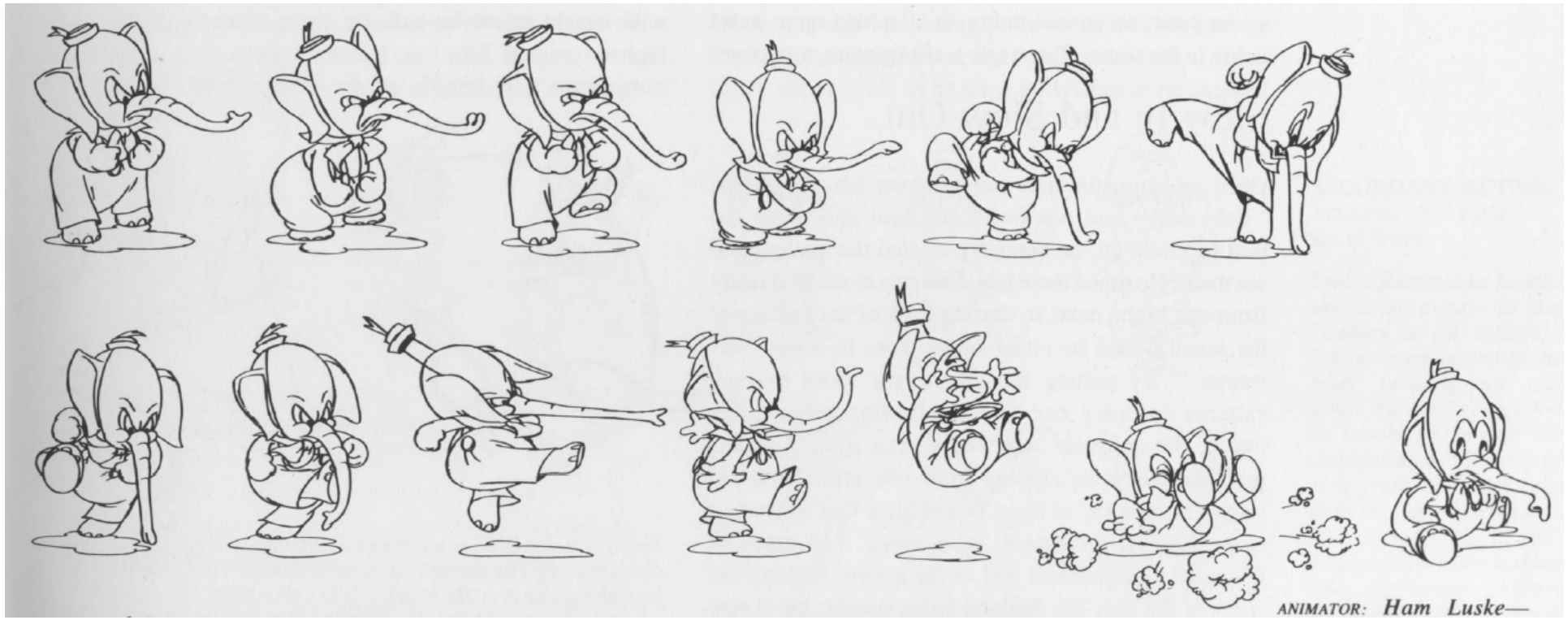
Follow Through and Overlapping

(Nachschwingen & Überschneidung)

Bei der Beendigung einer Bewegung bleibt kein Gegenstand einfach sofort stehen.

Hebt z.B. eine Person den Arm in die Höhe, hält die Armbewegung nach Erreichen der beabsichtigten Position nicht einfach an. Vielmehr bewegt sich der Arm zuerst etwas weiter, bevor er sich wieder ein wenig zurückzieht.

Nachschwingen



Exkurs

Weitere „klassische“ Animationsprinzipien

- **Slow in and Slow out**

Siehe Ende Abschnitt 6.2 „Keyframe Animation“

  - Animateur gibt die Schlüsselbilder vor
  - Um diese Schlüsselbilder zu betonen, sollen Bewegungen langsam von einem Schlüsselbild weg, schnell in den Zwischenbildern, und wieder langsam in das nächste Schlüsselbild erfolgen.

  → Die meiste Zeit der Animation entfällt auf die Schlüsselbilder (und was kurz davor und danach passiert).

- **Arcs**

Siehe Ende Abschnitt 6.2 „Keyframe Animation“

  - Bewegungen entlang gerader Linien sind nicht organisch.
  - Die meisten menschlichen Bewegungen erfolgen entlang gekrümmter Linien oder Bögen (Arcs)

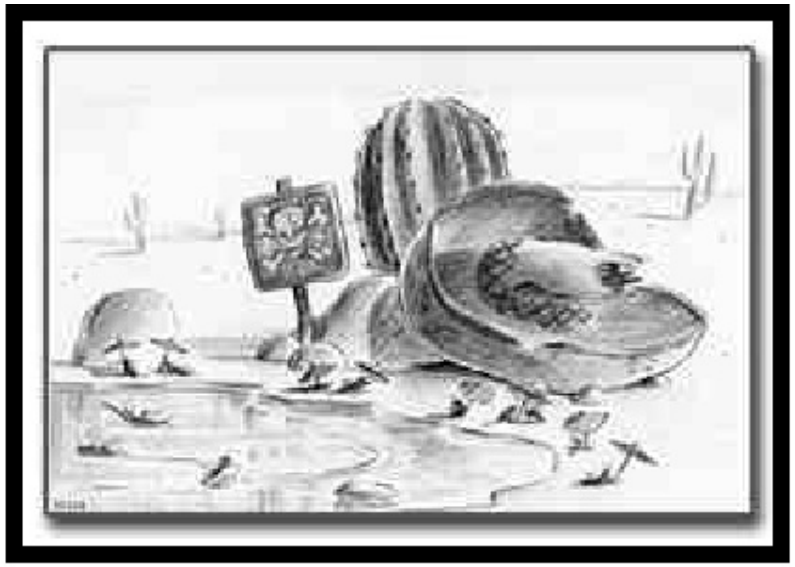
- **Solid Drawing and Appeal**
  - Animateure müssen zeichnen können, bevor sie Animationen erstellen können.
  - Ansprechendes Aussehen (Appeal) fesselt die Zuschauer. Selbst der Bösewicht kann „interessant“ aussehen.

- **Exaggeration** (Übertreibung, Überzeichnen)
  - Walt Disney wollte glaubhafte, überzeugende Übertreibung.
  - Falls eine Figur traurig ist, stell sie noch trauriger dar, wenn sie müde ist, stell sie noch müder dar.

Traditionelle Herstellung eines Animationsfilms

1. **Beschreibung der Story**
   - Story Board
   - Folge von Zeichnungen mit Beschreibungen
2. **Beschreibung der Bewegungen**
   - Schlüsselbilder (Key Frames)
   - Zeichne einige, wichtige Bilder als Linienzeichnungen, z.B. Anfang und Ende eines Schritts
3. **Zwischenbilder (Inbetweens)**
   - Zeichne restliche Bilder (als Linienzeichnungen)
   - Die Zeichner hierfür verdienen wenig.
4. **Ausmalen**
   - Übertrage auf Zelluloid und male farbig aus.
   - Die Zeichner hierfür verdienen noch weniger.

Story Board (hier: „A bug's life", „Das große Krabbeln")











For *Zootopia*, there were **197,136** story panels created. The boards are edited together with scratch dialogue (the voices of those who work on the movie, not the voice actors), providing suggested pacing for a given scene.

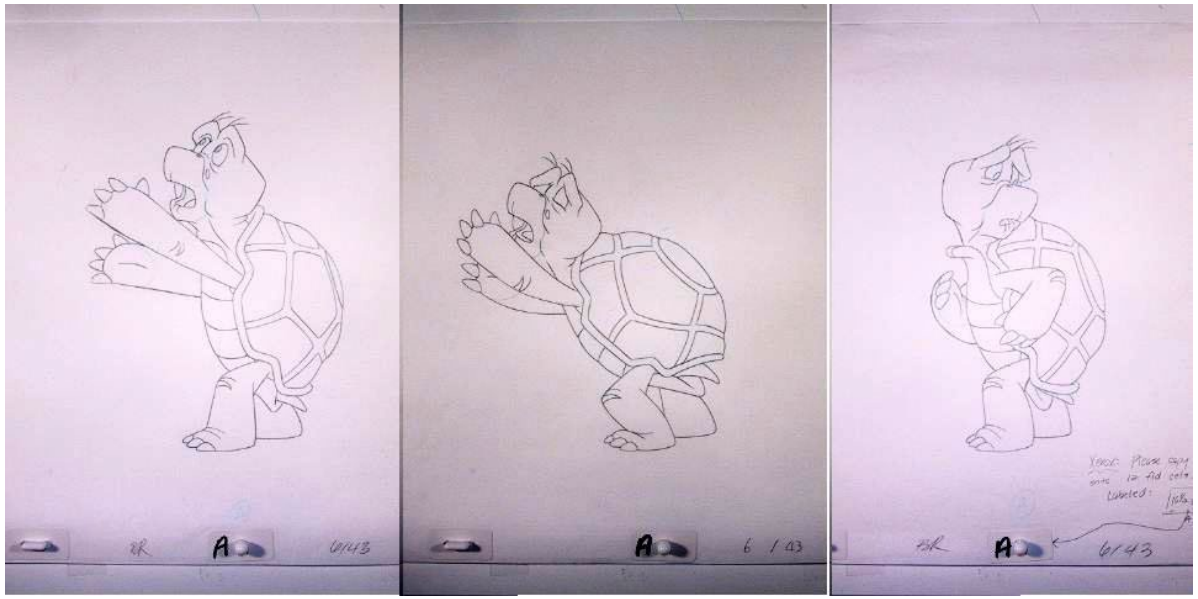
*deutscher Titel: Zoomania



https://www.youtube.com/watch?v=fmhr-9eRk3U

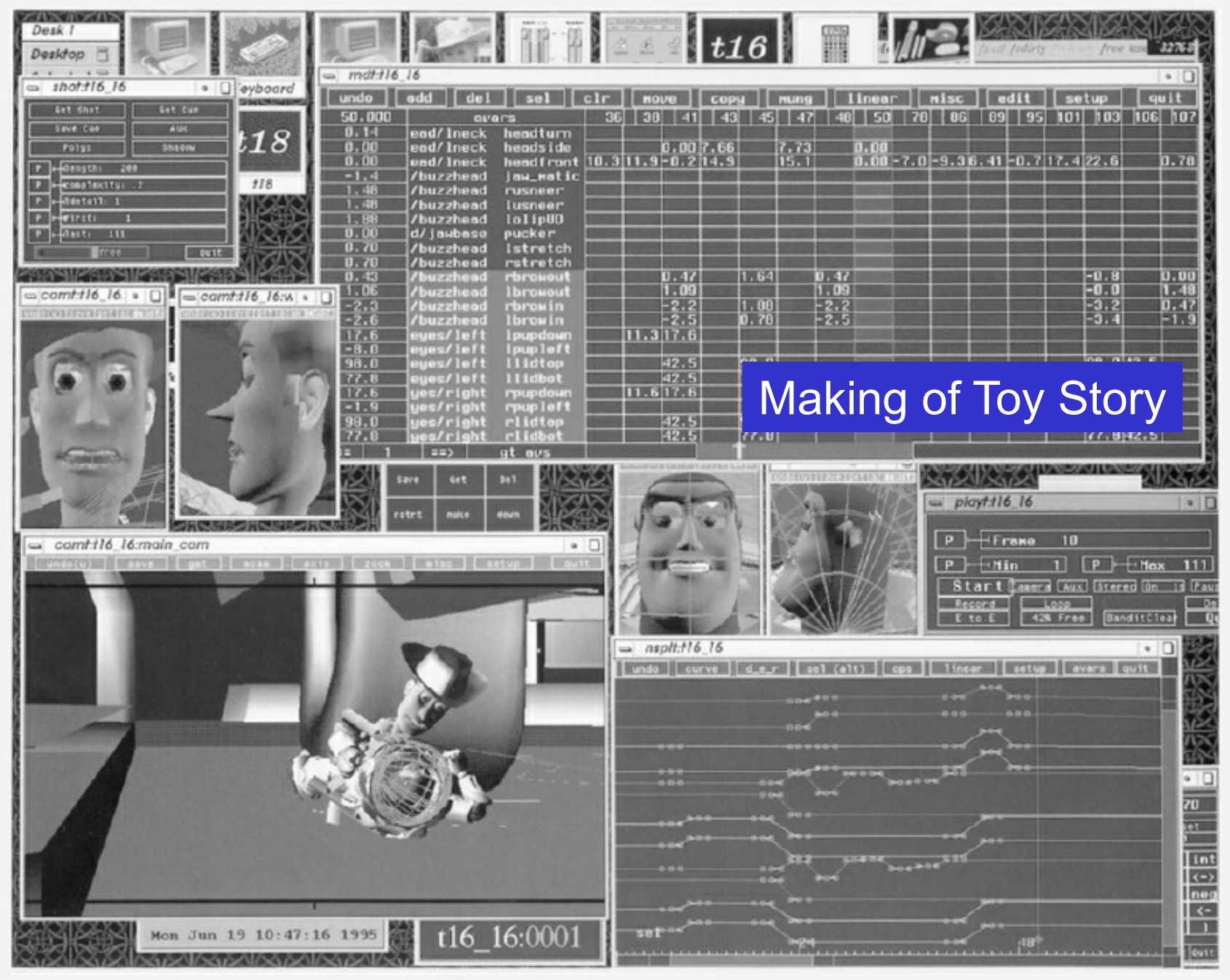
Herstellung eines Animationsfilms mit Hilfe von Computern

- Computerized Cel Painting
  - Zeichnungen werden eingescannt und dann vom Computer auf Zelluloid übertragen und ausgemalt. (z.B. König der Löwen)
- Cartoon Inbetweening
  - Automatische Interpolation zwischen zwei (eingescannten) Zeichnungen
  - Interpolation schwer
- Computeranimation
  - Erzeugen der Bilder durch Rendering von 3D-Modellen
  - automatisches Inbetweening





Weitere Beispiele:
http://www.elecorn.com/tweenmaker/gallery.html

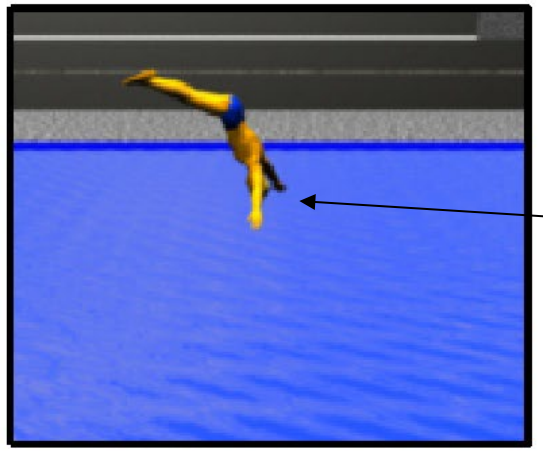


Computer-animation

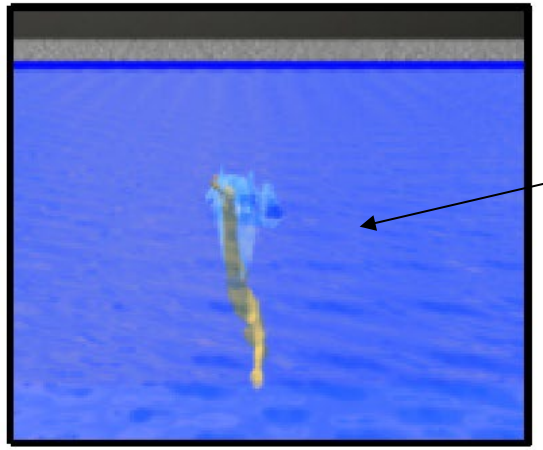
Was kann animiert werden ?

- Position und Ausrichtung
- Geometrie (Shape) and Größe
- Beleuchtung
- Reflektion
- Kamera

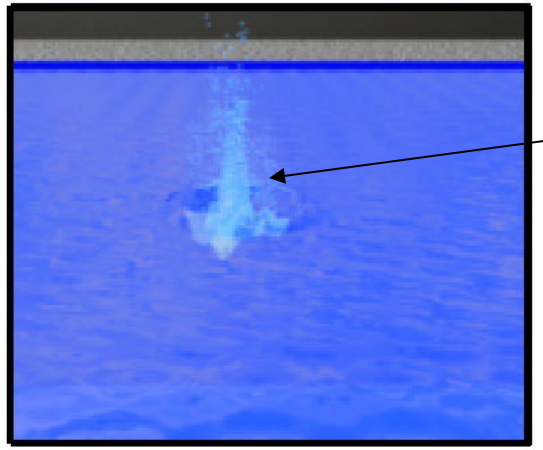
- … einfach alle Eigenschaften einer Szene



artikuliertes Objekt



deformierbares Objekt



Partikelsystem

→ Kombination verschiedener Modellierungstechniken

Vorlesung: Grundlagen der Computergrafik

Kapitel 6: Computer Animation



Kapitel 6.2:

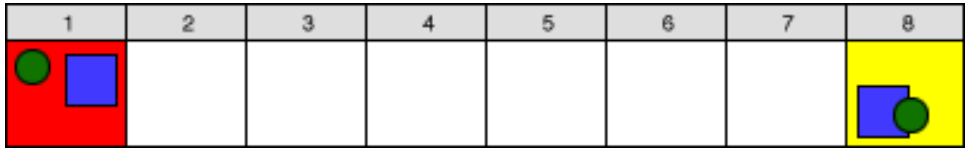
Keyframe Animation

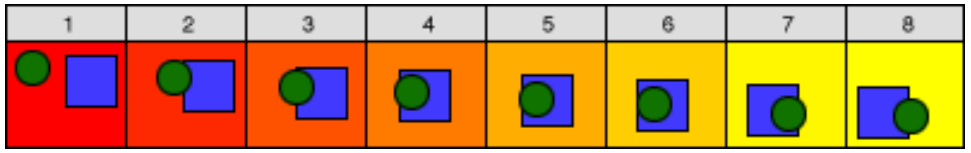
Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier

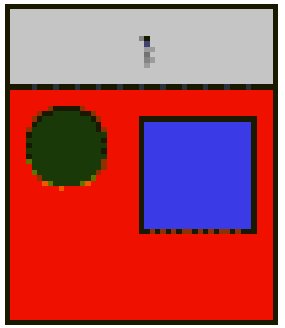


Keyframe Animation

- Jedes Schlüsselbild wird durch eine Menge von Schlüsselparametern (Zustand) spezifiziert
- Inbetweening: Interpoliert die Parameter für Zwischenbilder

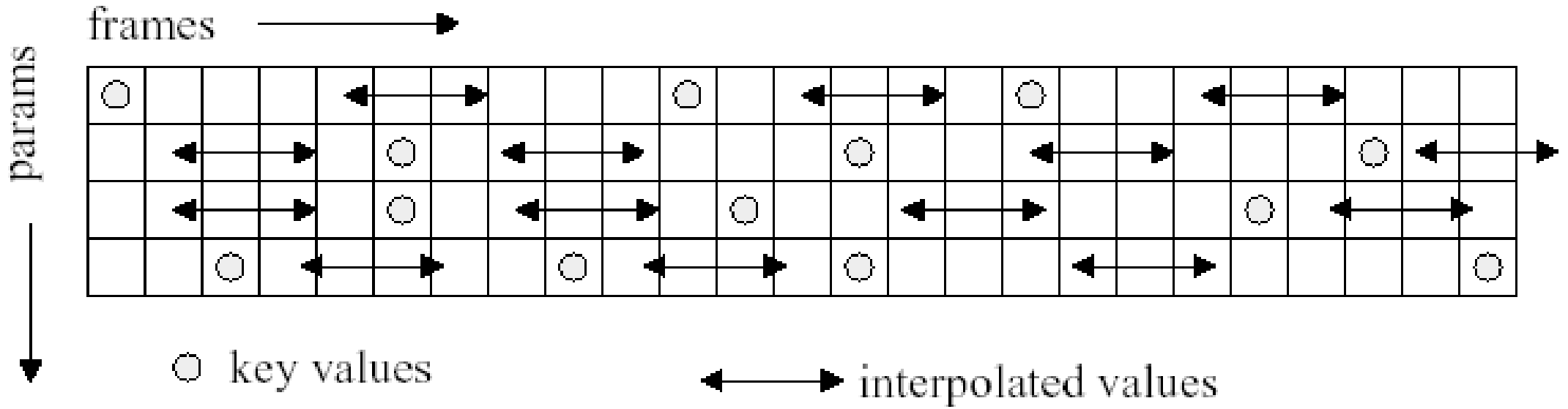




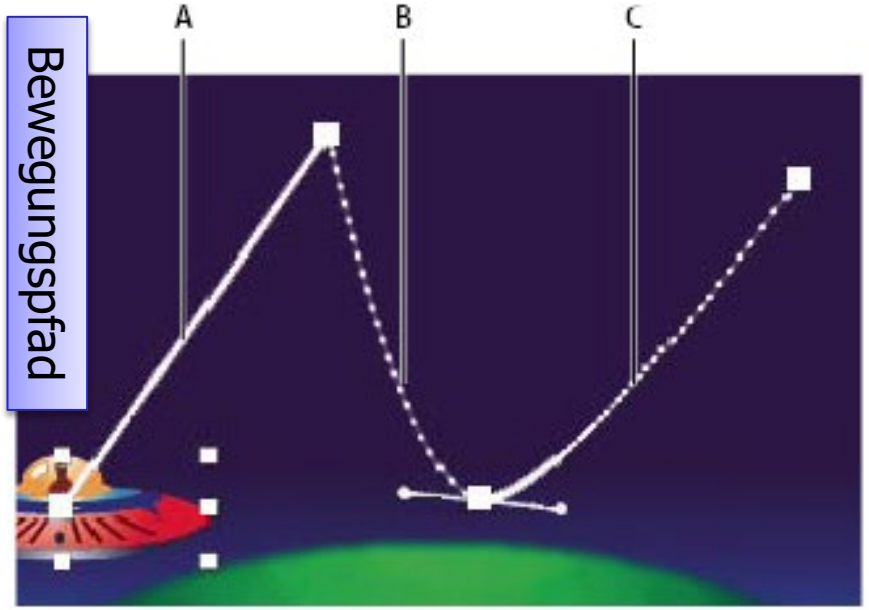


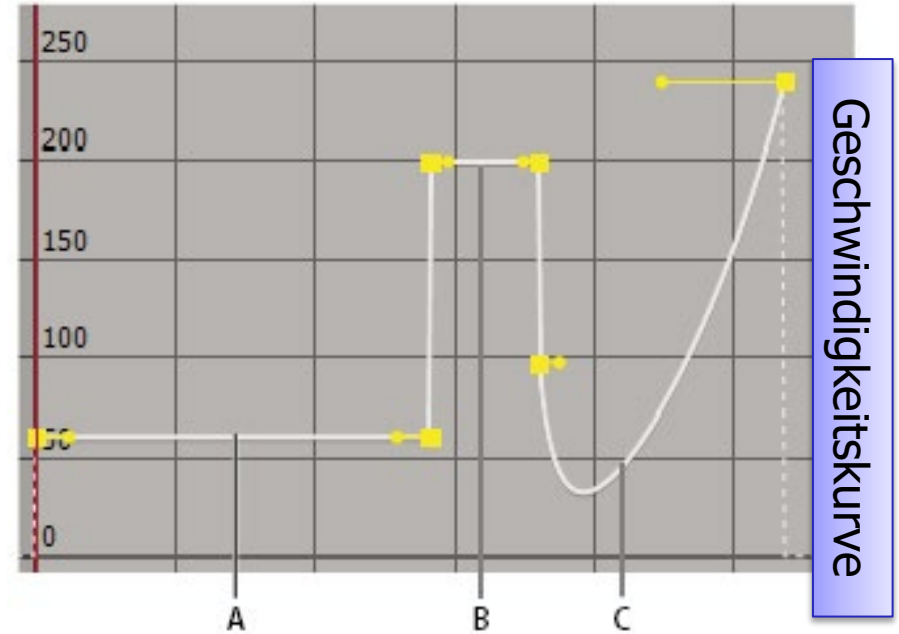
Keyframe Animation

- Im Gegensatz zu gezeichneten Zwischenbildern ist es möglich, jeden Schlüsselparameter nur an für diesen wichtigen Frames anzugeben.
- Der Computer **interpoliert** dann für jedes Zwischenframe die Werte jeden Parameters aus den Werten des Parameters in dessen Schlüsselframes.



Interpolation und Geschwindigkeitssteuerung





A. Punkte liegen nahe beieinander, wodurch eine geringere Geschwindigkeit angezeigt wird (oben); konstante Geschwindigkeit (unten).

B. Punkte liegen weit auseinander, wodurch eine höhere Geschwindigkeit angezeigt wird (oben); konstante Geschwindigkeit (unten).

C. Ungleichmäßige Anordnung der Punkte zeigt eine Geschwindigkeitsänderung an (oben); Geschwindigkeit sinkt und steigt dann wieder (unten).

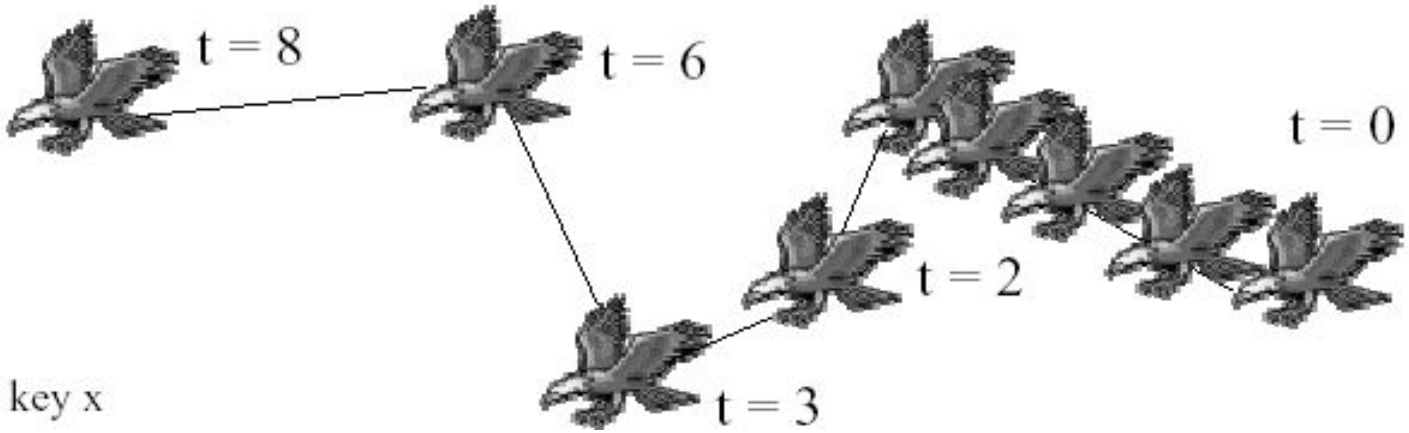
Quelle: https://helpx.adobe.com/de/after-effects/using/speed.html

Exkurs

Überspringen

Interpolation

- Lineare Interpolation
  - Nicht kontinuierlich, abrupte Wechsel
  - keine fließende Bewegung



Lineare Interpolation von t_0 bis t_1

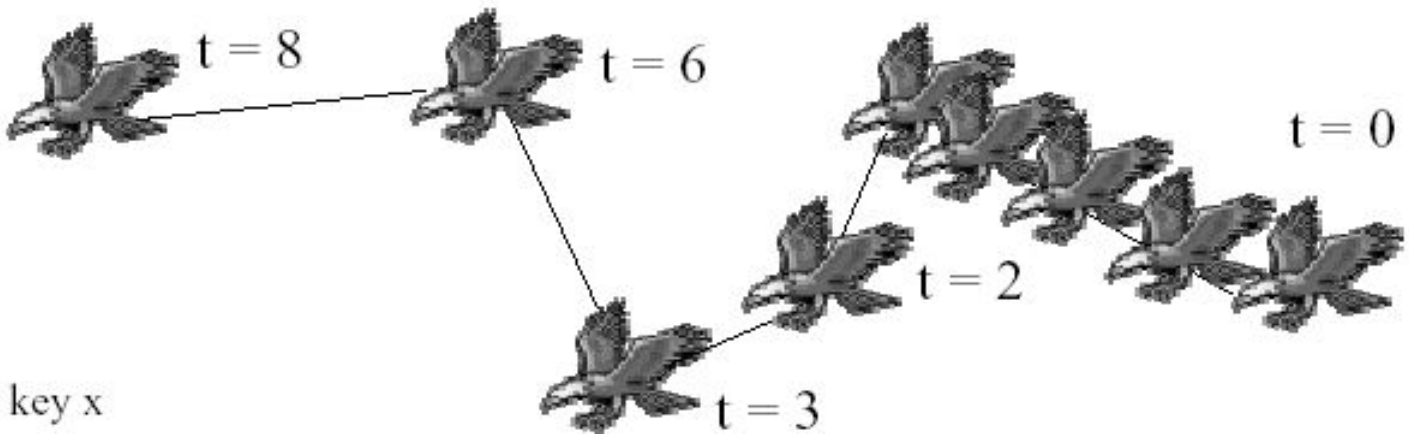
Startwert

$$x(t) = x_0 + \frac{t - t_0}{t_1 - t_0}(x_1 - x_0)$$

Zwischenwert

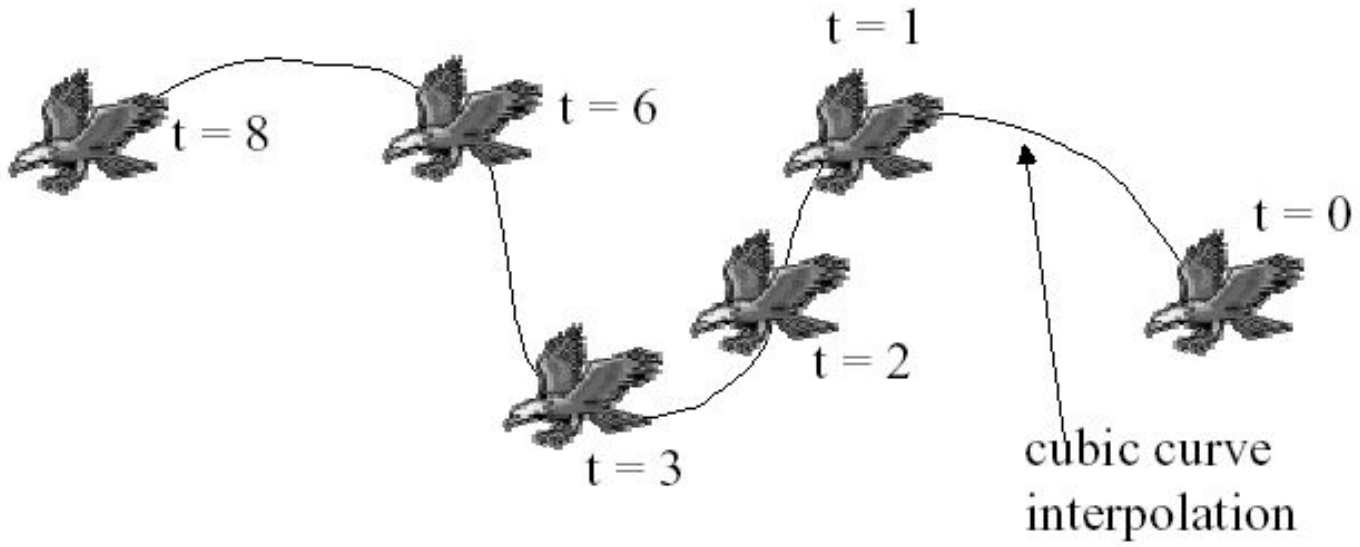
„Anteil“ des Teilstücks

Abstand zwischen Start- und Endwert



Interpolation

- Spline-Interpolation (wie ein Biegestab)
  - Fließende Bewegung, keine abrupten Richtungswechsel
  - Aber möglicherweise unerwünschte Effekte
    - Überschneidungen mit anderen Objekten

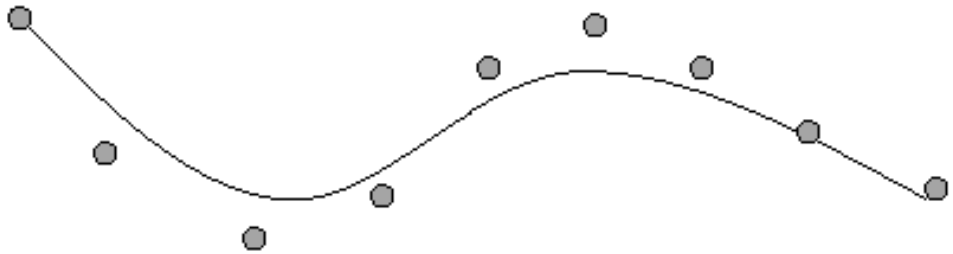


Interpolieren oder Approximieren?

- Interpolierte Kurve geht **durch** die Punkte
- Hermite und Catmull-Rom interpolieren

- Approximierte Kurve verläuft **nahe** zwischen den Punkten
- Bezier- und B-Splines approximieren

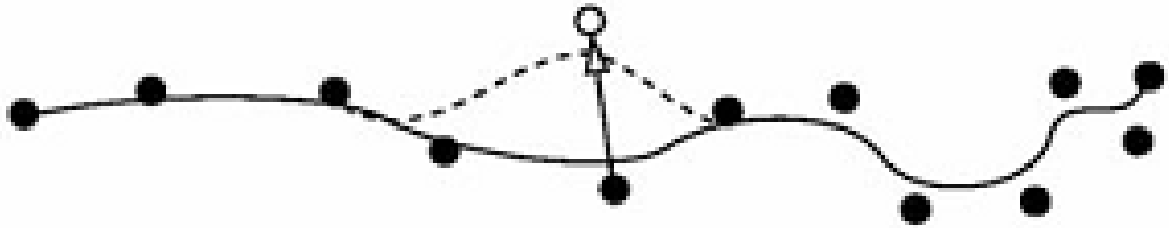




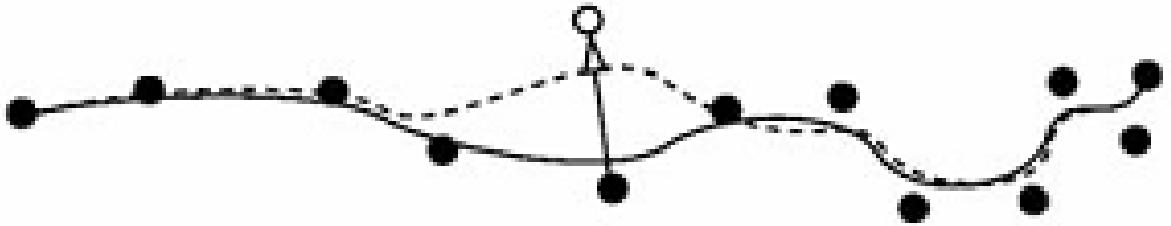
Exkurs

Globale oder Lokale Steuerung

- Beeinflusst eine kleine Änderung (Verschiebung eines Punktes) die gesamte Kurve oder nur ein Segment?
- Lokale Kontrolle ist meist intuitiver und wird z.B. durch zusammengesetzte Kurven ermöglicht.



a) local control: moving one control point only changes the curve over a finite bounded region



b) global control: moving one control point changes the entire curve; distant sections may change only slightly.

Exkurs

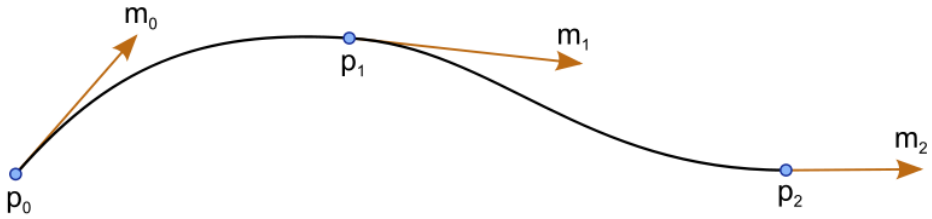
Beispiel: Hermite-Kurven

- kubische Polynomialfunktion mit $u \in [0,1]$:

$$P(u) = a*u^3 + b*u^2 + c*u + d$$



- Hermite-Interpolation benötigt
  - Endpunkte
  - Tangenten an den Endpunkten (Tangente = 1. Ableitung)
- Um eine zusammengesetzte Kurve zu erzeugen, verwendet man die Endpunkte einer Kurve als Anfang der nächsten und verwendet den gleichen Tangentenvektor.



Exkurs

Beispiel: Hermite-Kurven

- Kubische Polynomialfunktion und ihre Ableitung:

$$P(u) = a*u^3 + b*u^2+c*u+d$$

$$P'(u) = 3a*u^2 + 2b*u+c$$

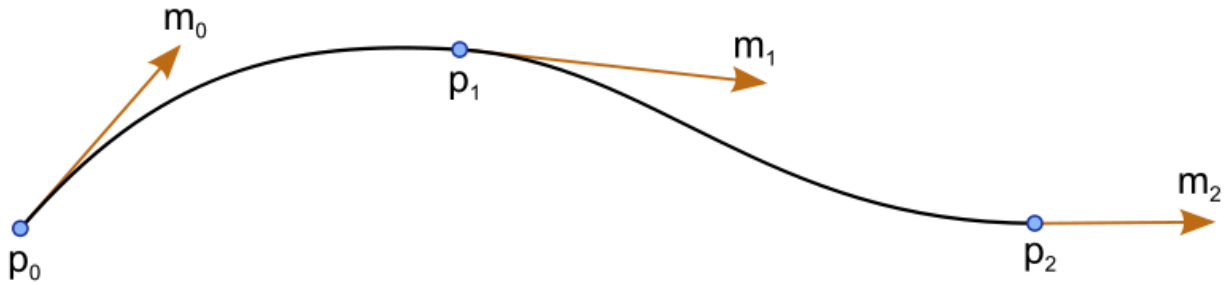
- Wir müssen a, b, c und d bestimmen. Gegeben sind dabei:
  - die Endpunkte: $P(0)=p_0$, $P(1)=p_1$
  - die Tangenten: $P'(0) =m_0$, $P'(1)=m_1$
- Lösung:

$$a = 2\,(P(0)-P(1))+P'(0)+P'(1)$$

$$b = 3\,(P(1)-P(0))-2P'(0)-P'(1)$$

$$c = P'(0)$$

$$d = P(0)$$



Exkurs

Hermite-Kurven in 3D

- Kubische Polynomialfunktion

$$P(u) = (x(u), y(u), z(u))$$

mit

$$x(u) = a_x*u^3 + b_x*u^2+c_x*u+d_x$$
$$y(u) = a_y*u^3 + b_y*u^2+c_y*u+d_y$$
$$z(u) = a_z*u^3 + b_z*u^2+c_z*u+d_z$$

Exkurs

Interpolation – Geschwindigkeit

- Steuerung der Geschwindigkeit
  - Es ist oft realistischer, eine Bewegung langsam zu starten, schneller zu werden, und dann wieder langsamer. (slow in, slow out)
  - Verwende Geschwindigkeitskurve (speed curve)
    - Geschwindigkeitskurve verbindet Zeit mit der Position auf dem interpolierten Spline.
    - Position auf dem interpolierten Spline bestimmt den Wert des animierten Parameters.

Exkurs

Wenn man den Parameter u mit gleicher Schrittweite erhöht, haben die Punkte (u,P(u)) unterschiedliche Distanz voneinander.

Wenn man also z.B. will, dass sich ein Objekt mit konstanter Geschwindigkeit entlang der Kurve bewegt, muss man u in Abhängigkeit von der Zeit berechnen u=u(t).

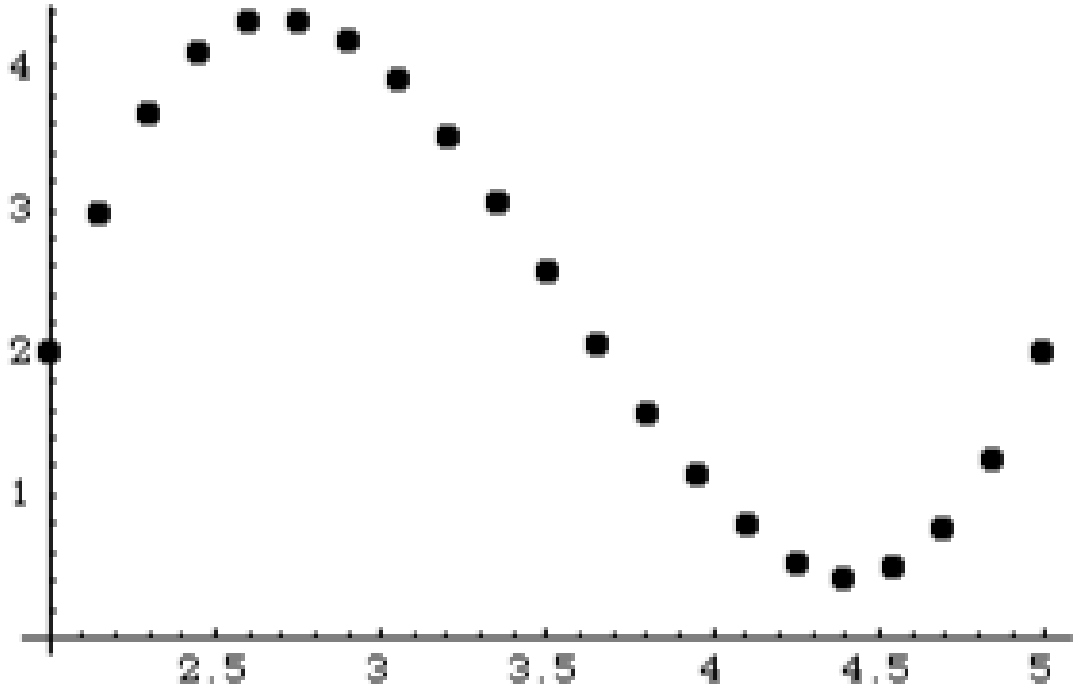


FIGURE 31. Example of points produced by equal increments of an interpolating parameter for a typcial cubic curve.

Exkurs

Geschwindigkeitssteuerung

- Kubische Polynomialfunktion:

$$P(u) = (x(u), y(u), z(u))$$

- Der Parameter u ist nicht die Zeit, sondern hängt von dieser ab, also $u(t)$
- Typische Vorgehensweise bei Keyframing
  - Zuerst wird der Pfad $P(u)$ entworfen.
  - Die Geschwindigkeitsfunktion $u(t)$ mit der genauen Geschwindigkeit und Timing wird anschließend entworfen.

Exkurs

Geschwindigkeitssteuerung

- Berechnung von $u(t) \in [0,1]$ mit $t \in [0,1]$:
- über die Bogenlänge u(s) bei Keyframing:

  $LENGTH(u)$ = Länge entlang der Hermitekurve von $(0,P(0))$ nach $(u,P(u))$
- Bei linearer Geschwindigkeit:

  $u(t) = u' \; mit \; LENGTH(u') = t*S$

  $mit \; S = LENGTH(1) = Gesamtlänge \; der \; Kurve$
- Zusätzliche Geschwindkeitssteuerung s(t): $P(u(s(t)))$

Exkurs

P(u(s(t)))

Relevant für **Aufgabe 1** von **Übungsblatt 11**

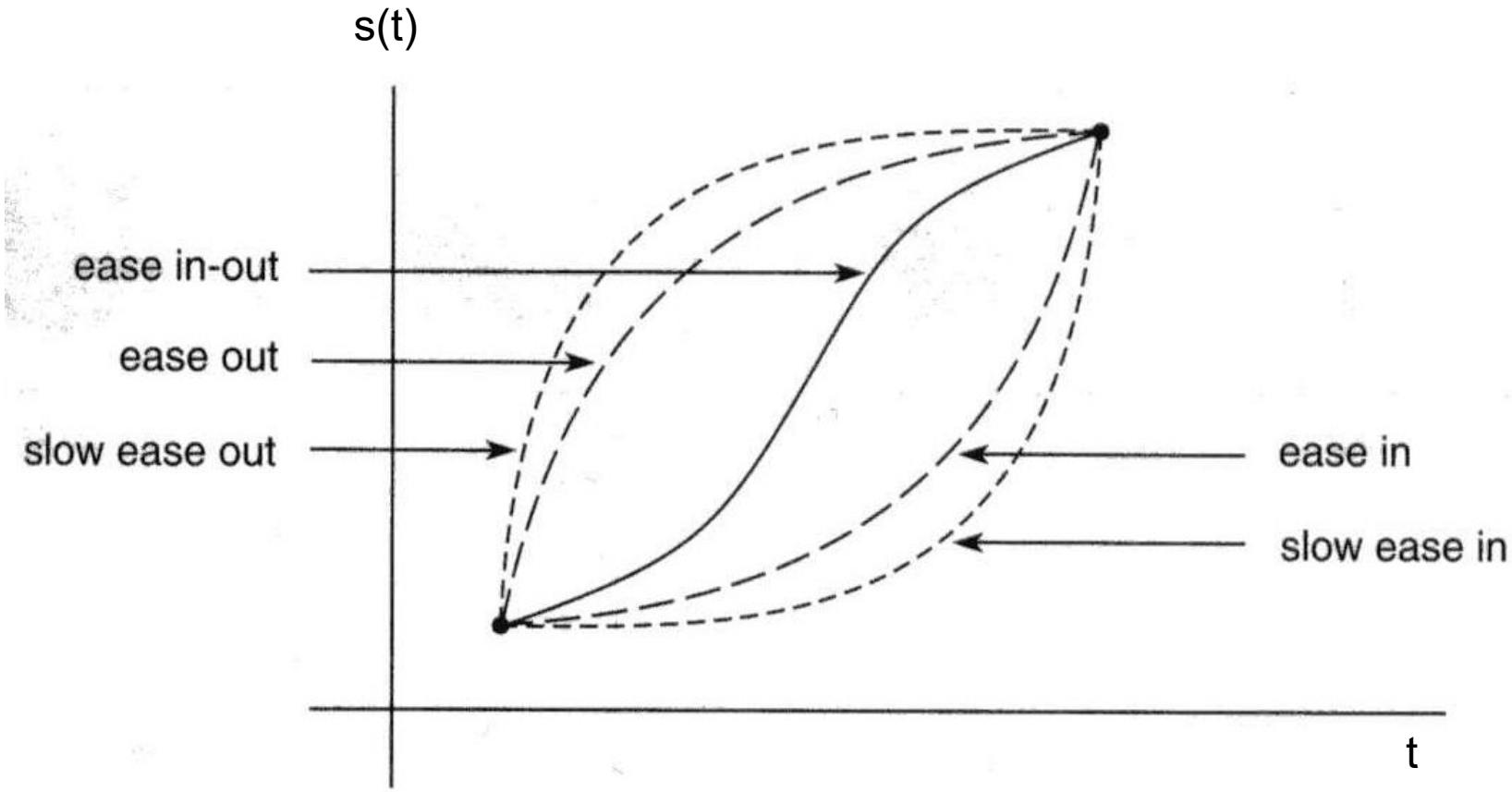
- p = P(u) : Parametrische Kurvengleichung
- u = u(s) : Parameter in Abhängigkeit der Kurvenlänge
- s = s(t) : Kurvenlänge in Abhängigkeit der Zeit

- s(t) ist der gewünschte Geschwindigkeitsverlauf
- u(t) gleicht Geschwindigkeitsschwankungen aufgrund des Kurvenverlaufs aus
- P(u) ist der Punkt der Kurve für den Parameter u.

Exkurs

Relevant für **Aufgabe 1** von **Übungsblatt 11**

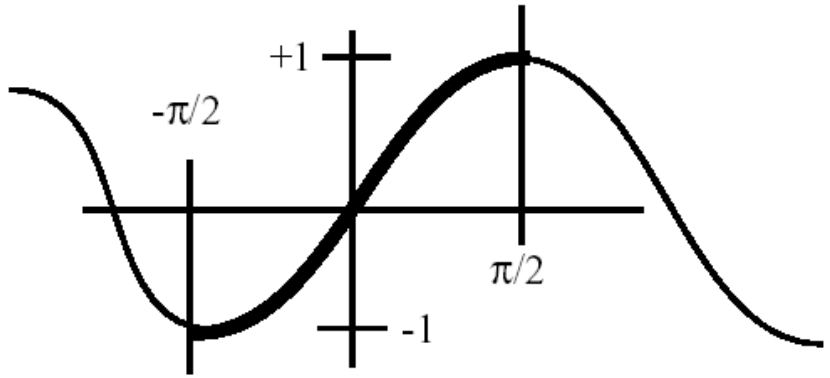
Geschwindigkeitssteuerung: $P(u(s(t)))$



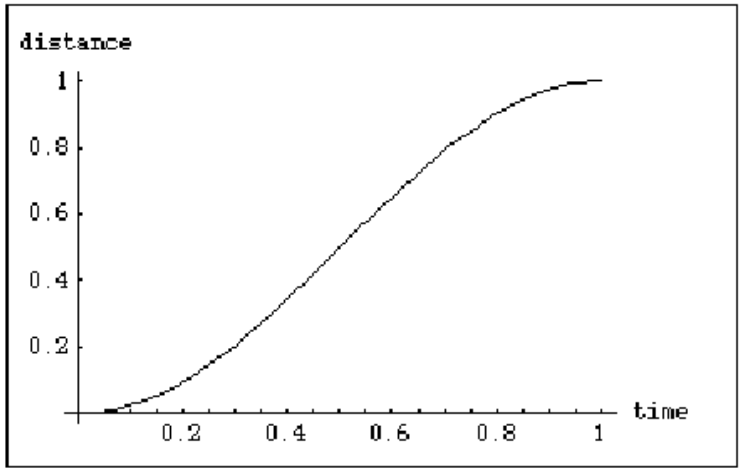
Exkurs

Relevant für **Aufgabe 1** von **Übungsblatt 11**

Slow in/slow out mittels Sinus



a) Sine curve segment to use as ease-in/ease-out control



b) Sine curve segment mapped to useful values

$$s(t) = ease(t) = \frac{\sin\left(t\pi - \frac{\pi}{2}\right) + 1}{2}$$

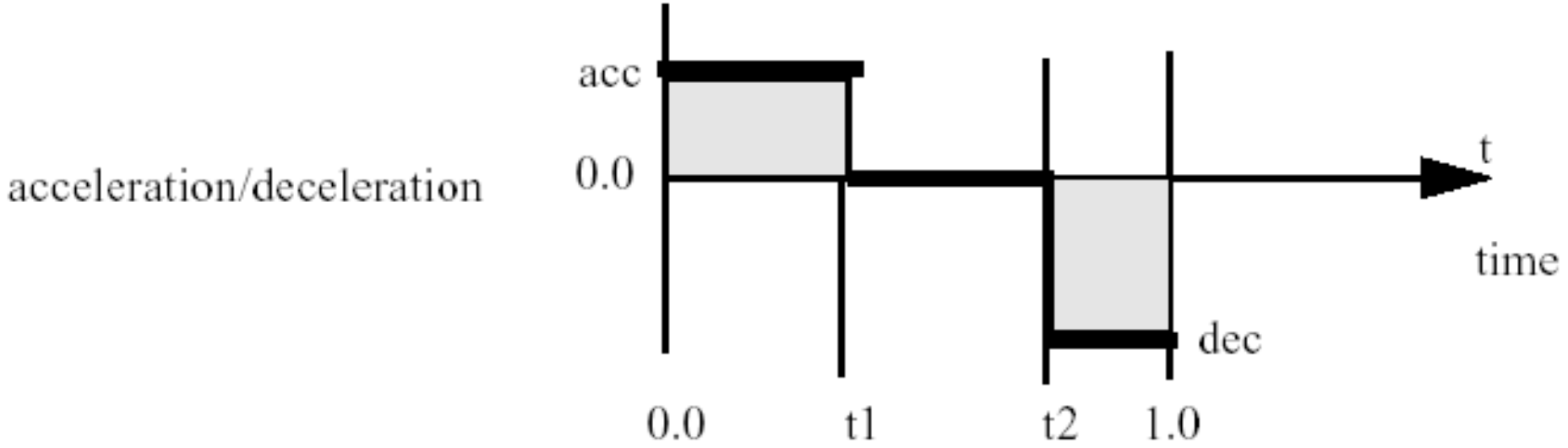
$$t = 0.25 \rightarrow s(t) = 0.1465$$

$$t = 0.75 \rightarrow s(t) = 0.8535$$

Exkurs

Relevant für **Aufgabe 1** von **Übungsblatt 11**

Physikalische Simulation:
Konstante Beschleunigung für parabolisches Slow-In/Slow-Out



$$a = acc \qquad 0 < t < t1$$
$$a = 0.0 \qquad t1 < t < t2$$
$$a = dec \qquad t2 < t < 1.0$$

Exkurs

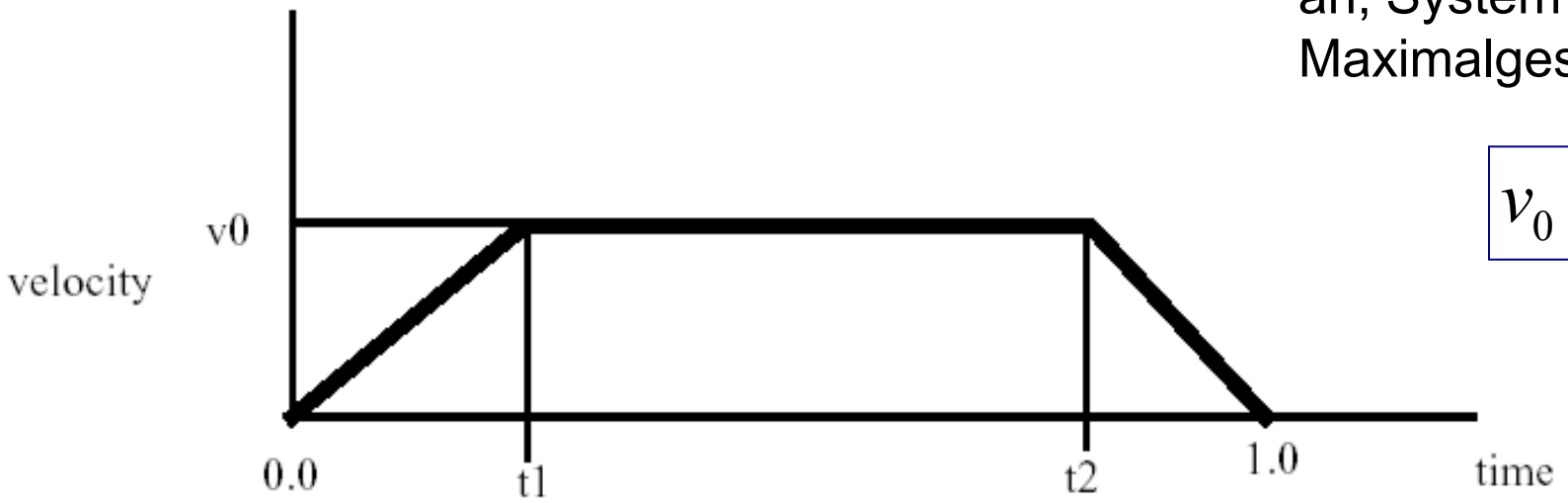
Relevant für **Aufgabe 1** von **Übungsblatt 11**

Konstante Beschleunigung für parabolisches Slow-In/Slow-Out

- Integrieren liefert Geschwindigkeit:

Benutzer gibt t1 und t2 an, System berechnet Maximalgeschwindigkeit

$$v_0 = ?$$



$$v = v0 \cdot \frac{t}{t1} \qquad 0.0 < t < t1$$

$$v = v0 \qquad t1 < t < t2$$

$$v = v0 \cdot \left(1.0 - \frac{t - t2}{1.0 - t2}\right) \qquad t2 < t < 1.0$$

$$v_0 = a * t_1$$
$$\Rightarrow a = \frac{v_0}{t_1}$$

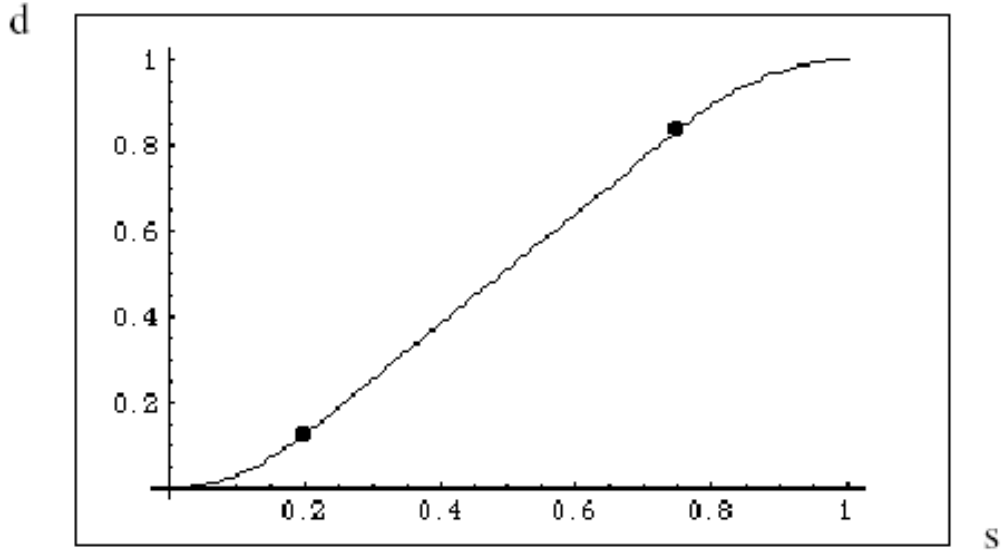
$$t1 = 1 - t_2$$

Exkurs

Relevant für **Aufgabe 1** von **Übungsblatt 11**

Konstante Beschleunigung für parabolisches Slow-In/Slow-Out

- Distanzfunktion (zurückgelegte Distanz) hat parabolische Abschnitte an beiden Enden



$$d = \frac{1}{2}at^2$$

$$a = \frac{v_0}{t_1}$$

$$d = v0 \cdot \frac{t^2}{2 \cdot t1} \qquad 0.0 < t < t1$$

$$d = v0 \cdot \frac{t1}{2} + v0 \cdot (t - t1) \qquad t1 < t < t2$$

$$d = v0 \cdot \frac{t1}{2} + v0 \cdot (t2 - t1) + \left(v_0 - \frac{\left(v_0 \cdot \frac{t - t_2}{1 - t_2} \right)}{2} \right) \cdot (t - t_2) \qquad t2 < t < 1.0$$

Exkurs

Relevant für **Aufgabe 1** von **Übungsblatt 11**

Durch Nachrechnen

- Für t=1.0 ist d=1

$$v_0 = \frac{2.0}{(t_2 - t_1 + 1.0)}$$

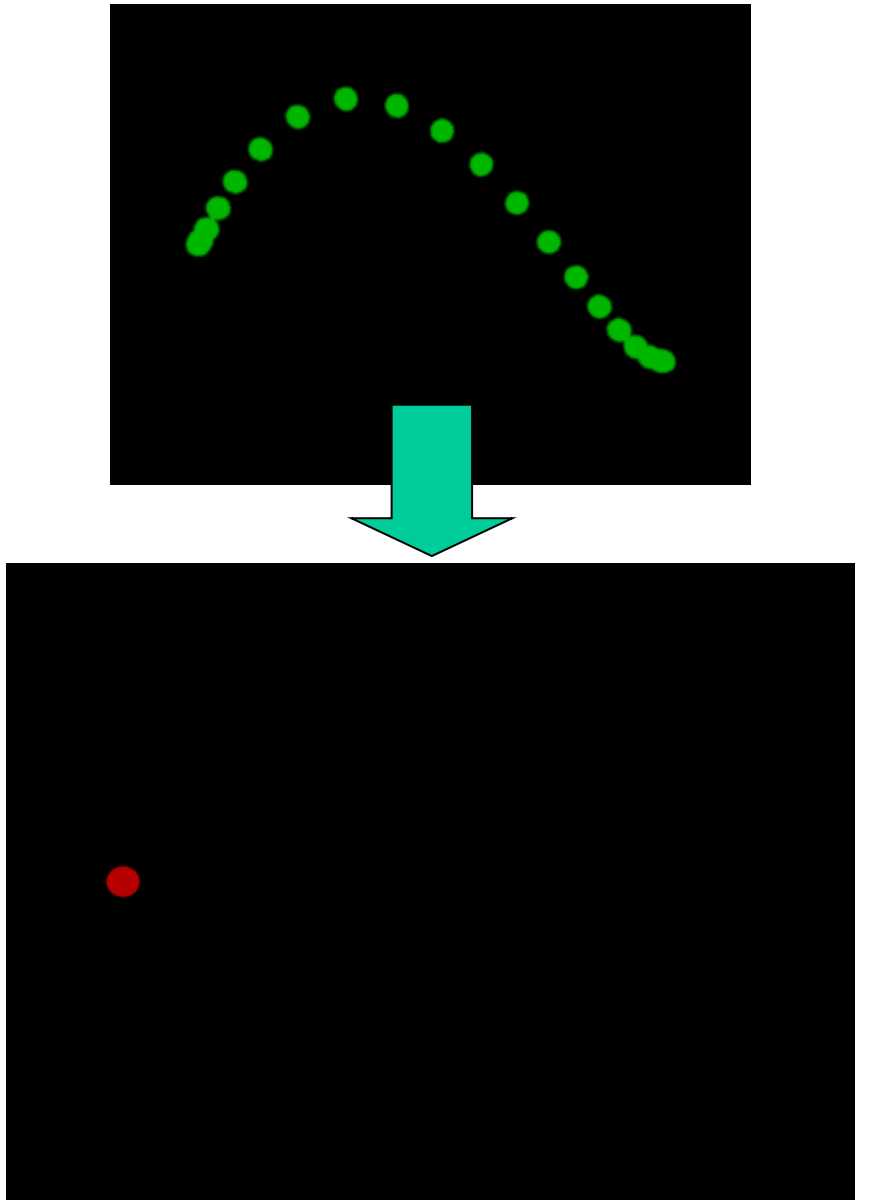
Exkurs

Relevant für **Aufgabe 1** von **Übungsblatt 11**

Parabolisches Slow-In/Slow-Out

```
double slowInOut(double t, double t1, double t2)
{  double v0;

   v0 = 2/(1+t2-t1);
   if (t<t1)  {
      d = v0*t*t/(2*t1);  }
   else {
      d = v0*t1/2;
      if (t<t2)  {
         d += (t-t1)*v0;  }
      else {
         d += (t2-t1)*v0;
         d += (t-t*t/2-t2+t2*b/2)*v0/(1-t2); }  }
   return (d);
}
```



Zusammenfassung: Keyframe-Animation

- Spezifikation der Schlüsselbilder/-parameter
  - Spezifiziere Schlüsselparameter und ihre Werte zu bestimmten Zeitpunkten
  - Spezifiziere Art der Interpolation
  - Spezifiziere Geschwindigkeitskurve
  - ➔Der Computer erzeugt die Zwischenbilder

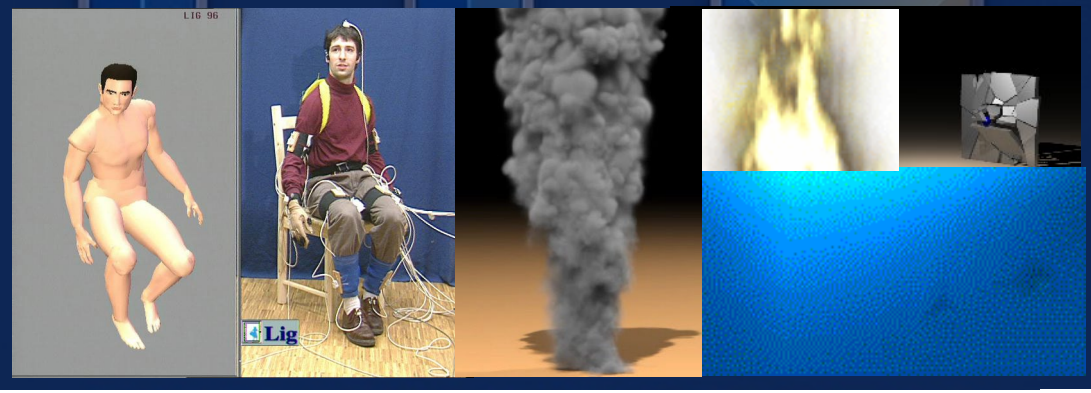
- 👍 Gute Kontrolle über die Bewegung
- 👍 Erleichterung gegenüber der traditionellen Animationsmethode, …
- 👎 … aber immer noch arbeitsaufwendig.
- 👎 Unbrauchbar für komplexe Szenen, bei denen sich alles bewegt: Gras im Wind, Wasser, Menschenmengen, Haare, …

Vorlesung: Grundlagen der Computergrafik

Kapitel 6: Computer Animation

Kapitel 6.3:

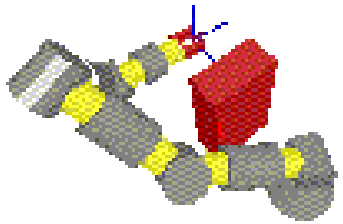
Andere Animationstechniken

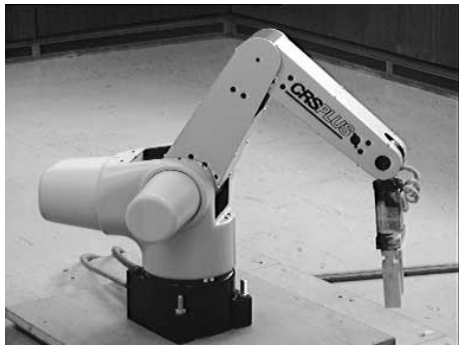


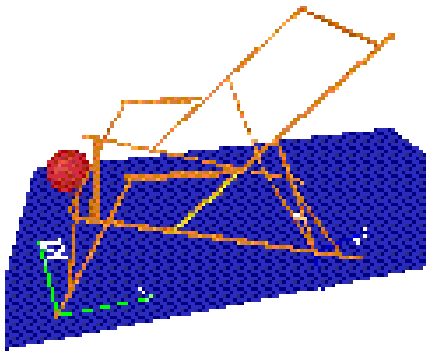
Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier

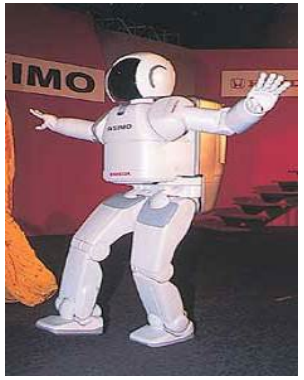
Chapter
6.3.1

Kinematik

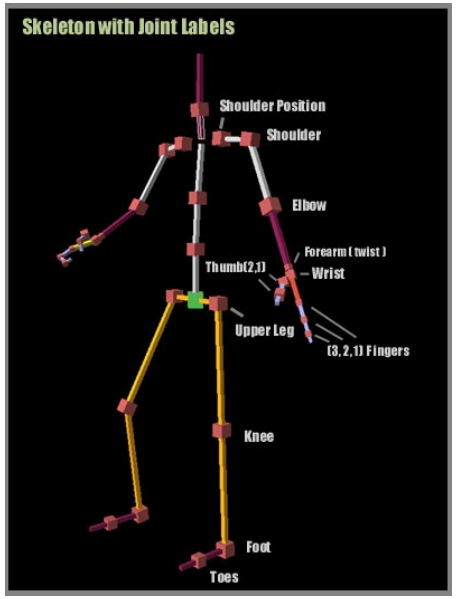






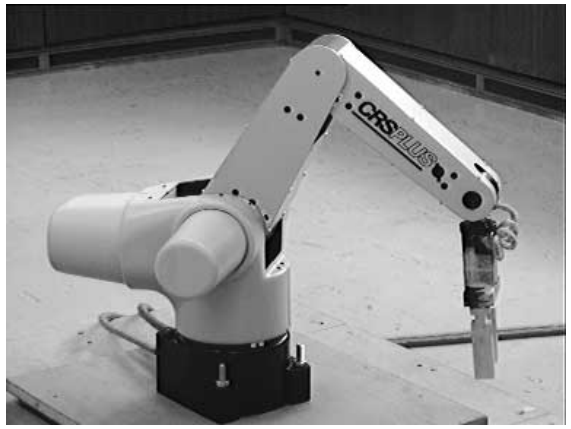


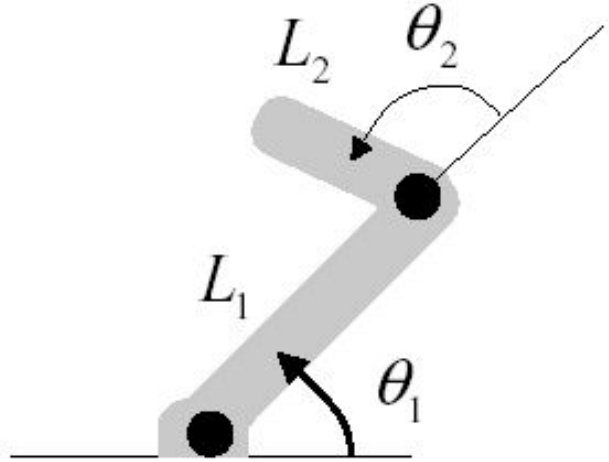




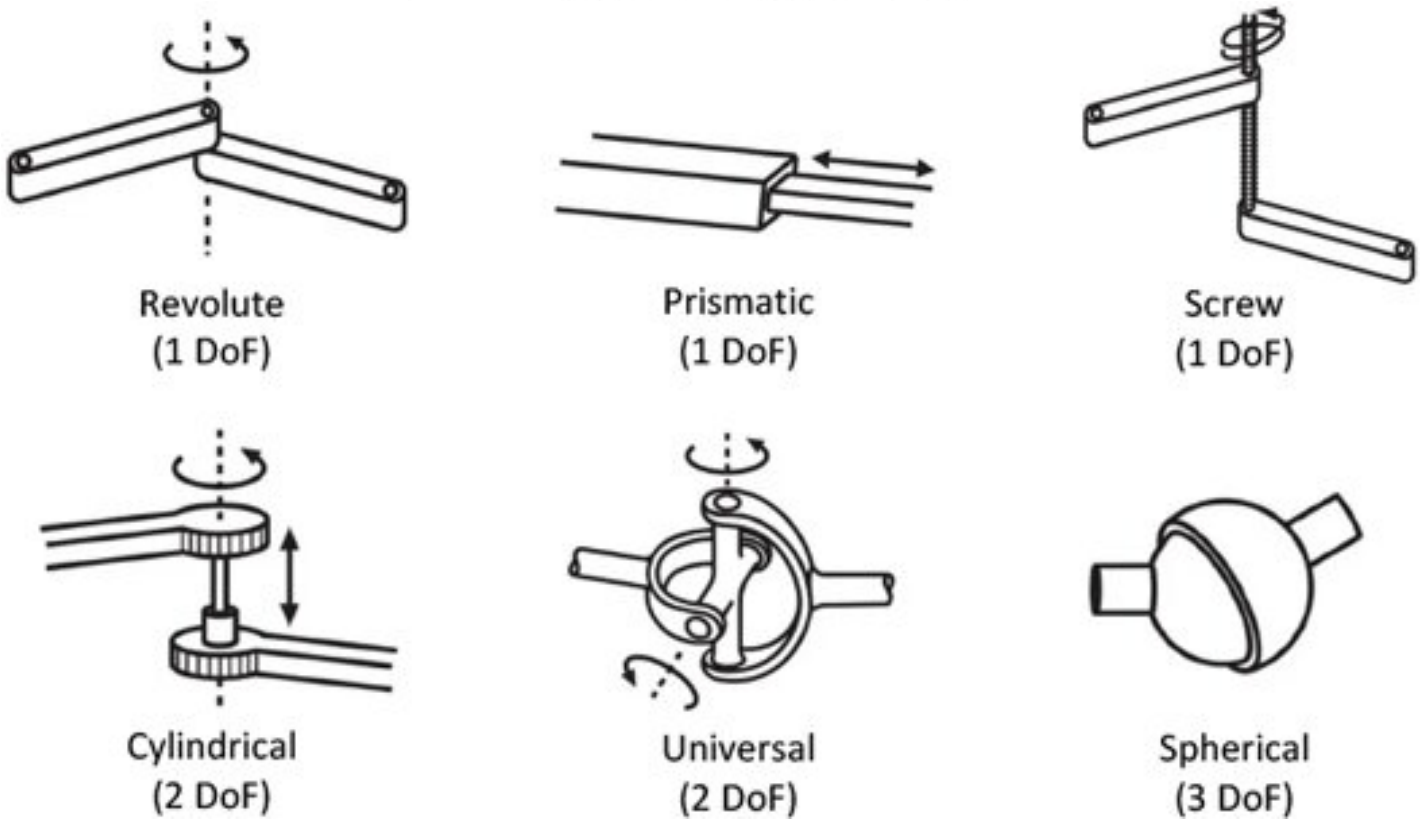
Animation beweglicher Strukturen

- im Englischen: **articulated structure**
- Objekt besteht aus Teilobjekten, die über Gelenke (**joints**) miteinander verbunden sind.
- Jedes Gelenk wird durch mindestens einen (Schlüssel-)Parameter kontrolliert
- Bewegungen werden durch Änderungen der Werte dieser Parameter erreicht.

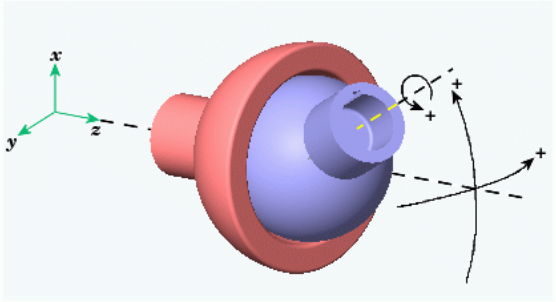


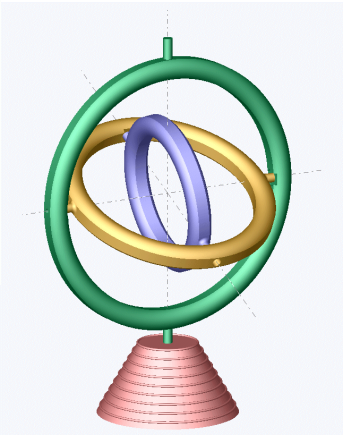


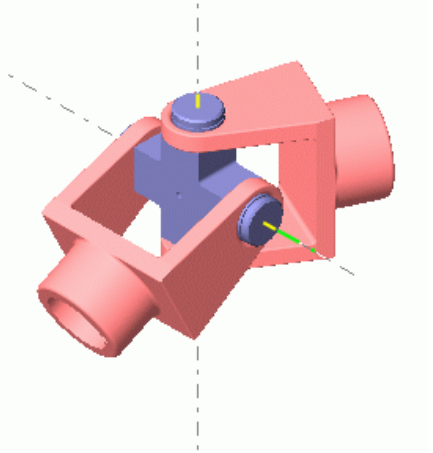
Arten von Gelenken



Die Parameter können durch weitere Abhängigkeiten eingeschränkt sein !!!

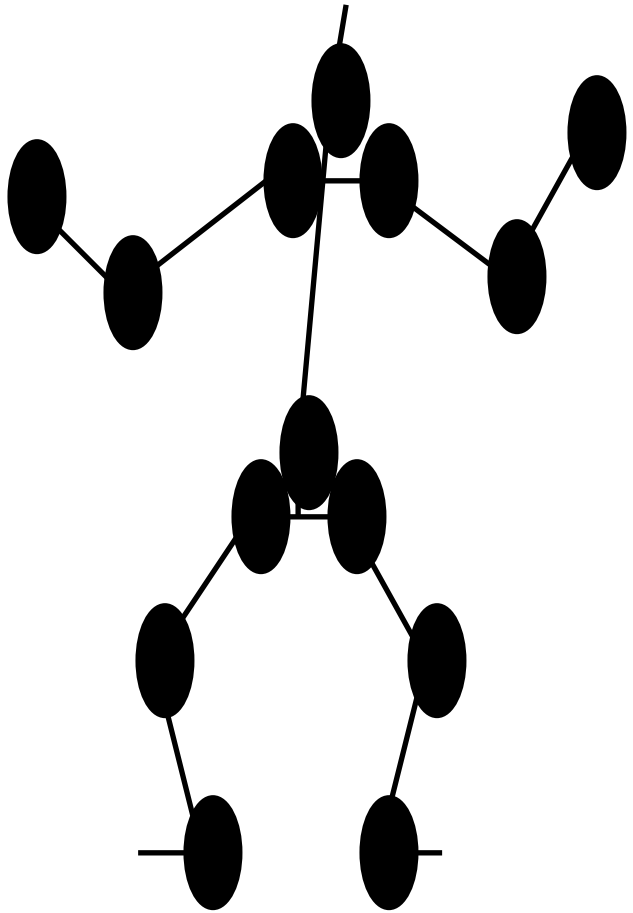






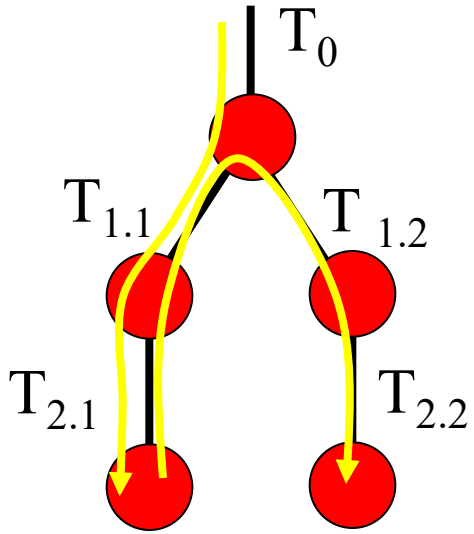
→ **Robotik**: Denavit-Hartenberg-Parameter und -Transformationen

Beispiel: Bewegliche Struktur



- Einfaches Skelett besteht aus 14 Gelenken
- Jedes Gelenk hat 2 oder 3 Freiheitsgrade
- Einige Parameter sind eingeschränkt.

Traversieren der Gelenkhierarchie



$$M = I$$

$$M = T_0$$

$$M = T_0 * T_{1.1}$$

$$M = T_0 * T_{1.1} * T_{2.1}$$

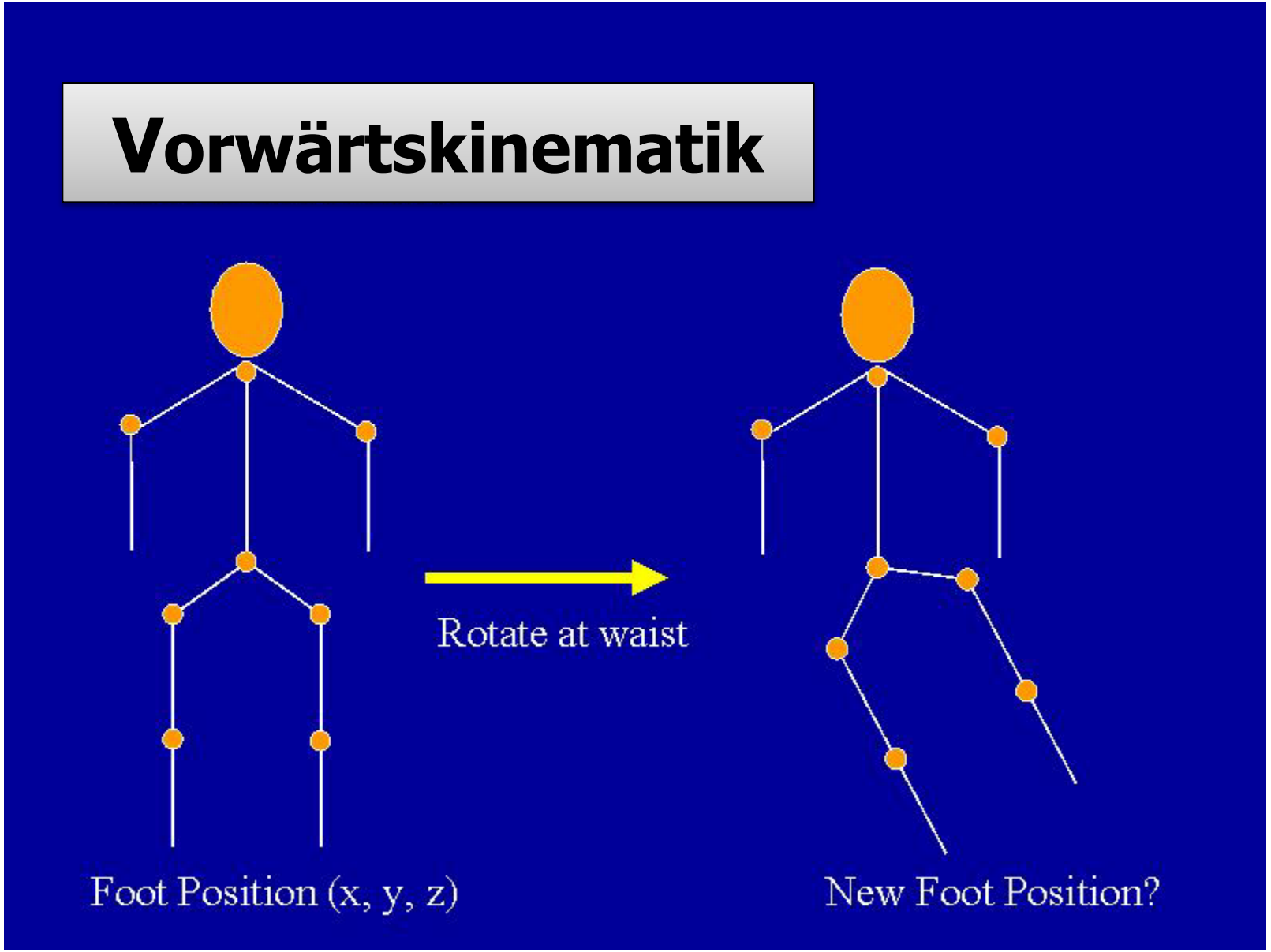
$$M = T_0 * T_{1.1}$$

$$M = T_0$$

$$M = T_0 * T_{1.2}$$

$$M = T_0 * T_{1.2} * T_{2.2}$$

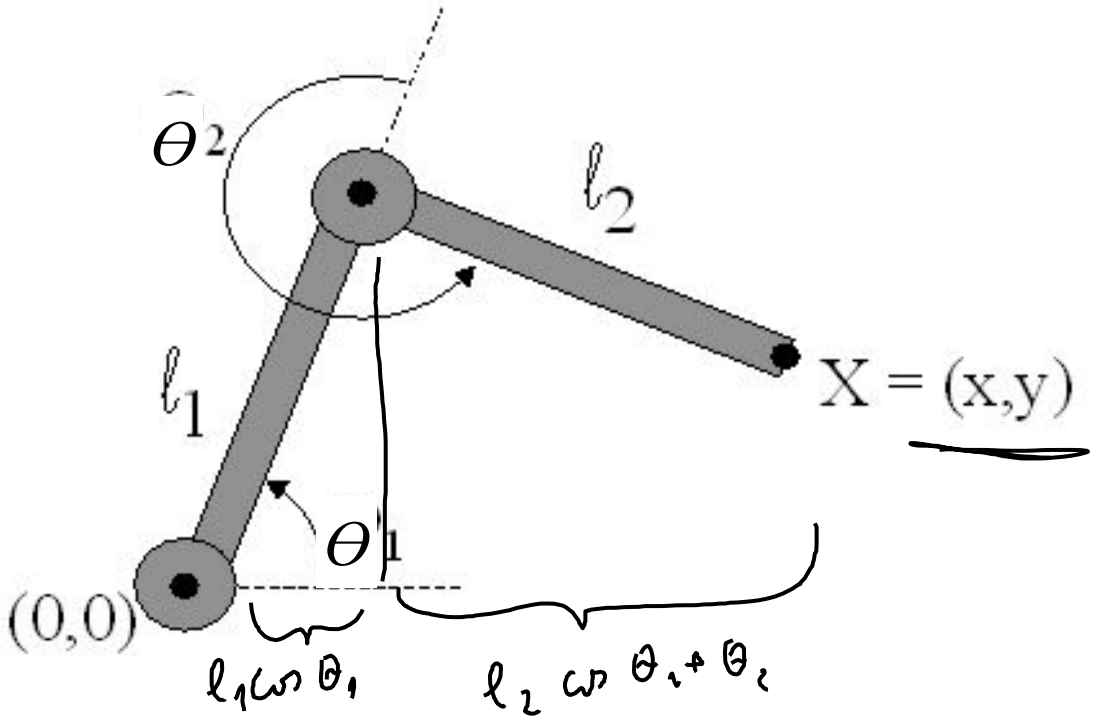
Vorwärtskinematik



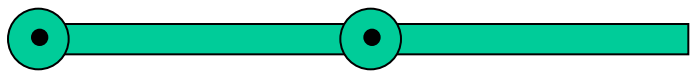
Vorwärtskinematik

- Animator setzt die Werte der Parameter für die Gelenke
- Der Computer berechnet Position und Ausrichtung der Verbindungen:

$$X = f(\theta_1, \theta_2)$$



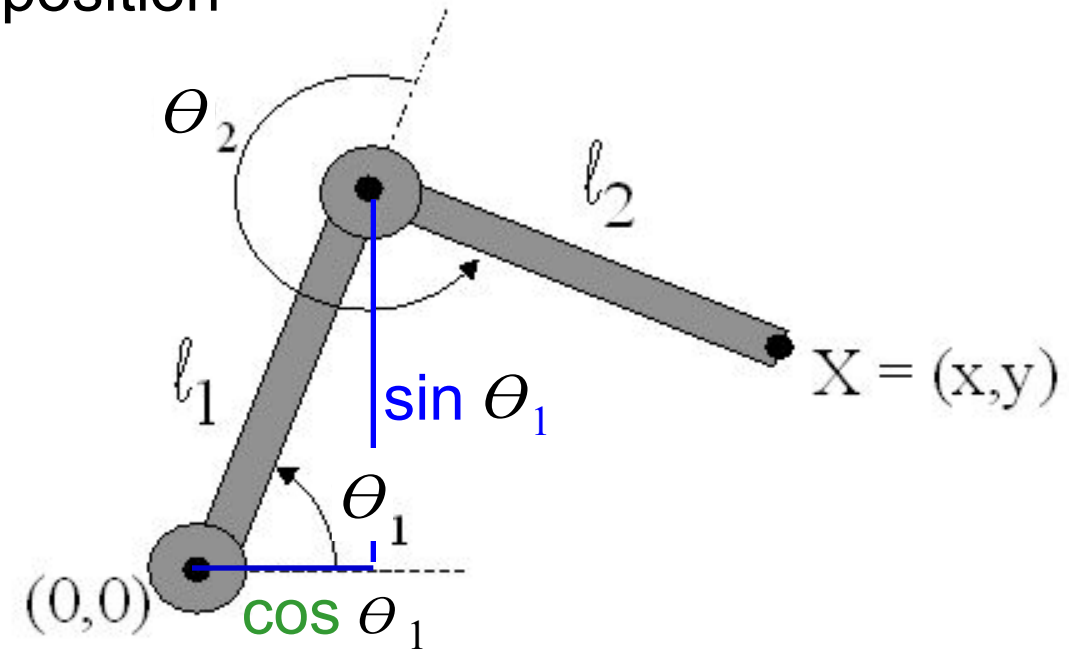
Vorwärtskinematik: Beispiel



$\theta_1 = \theta_2 = 0$

Gegeben: Winkel

Gesucht: Endposition

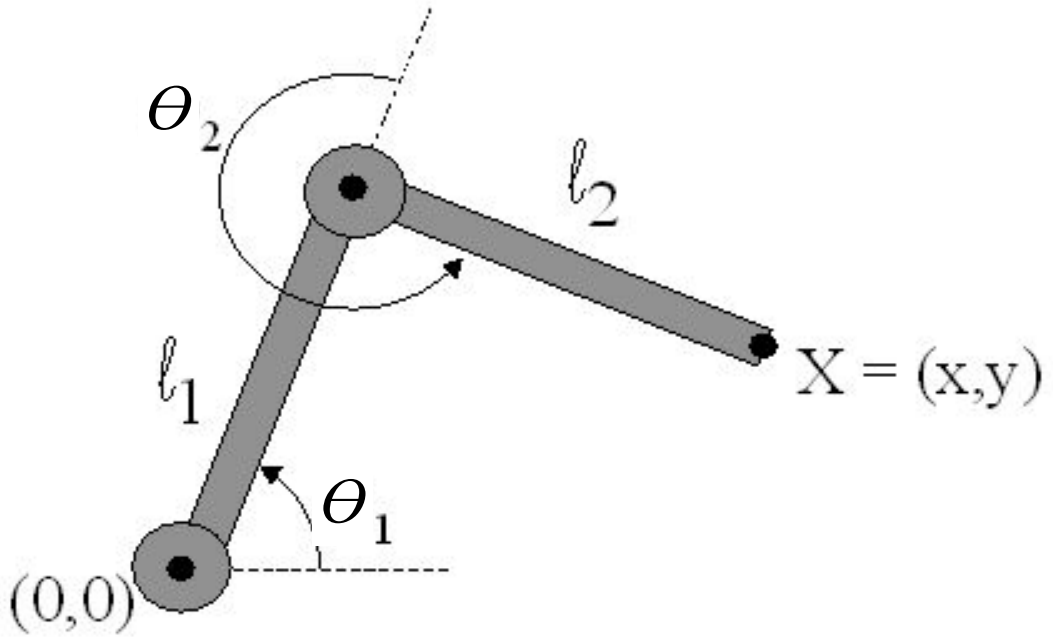


$$X = (l_1 cos\theta_1 + l_2 cos(\theta_1 + \theta_2), l_1 sin\theta_1 + l_2 sin(\theta_1 + \theta_2))$$

Vorwärtskinematik: Beispiel

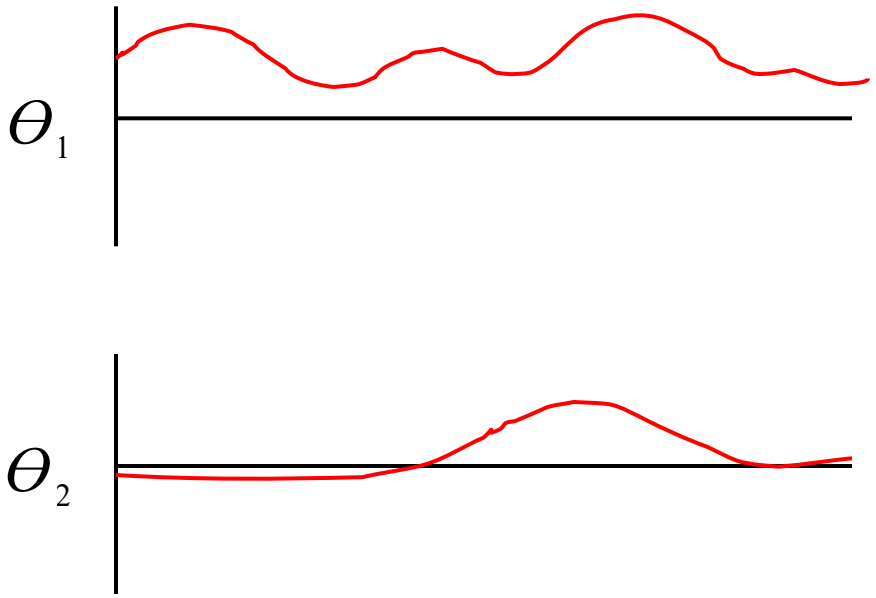
Einfacher mit Hilfe von Matrizen:

$$(x,y,w)^T = T(\ell_2)*R(\Theta_2)*T(\ell_1)*R(\Theta_1)*(0,0,1)^T$$



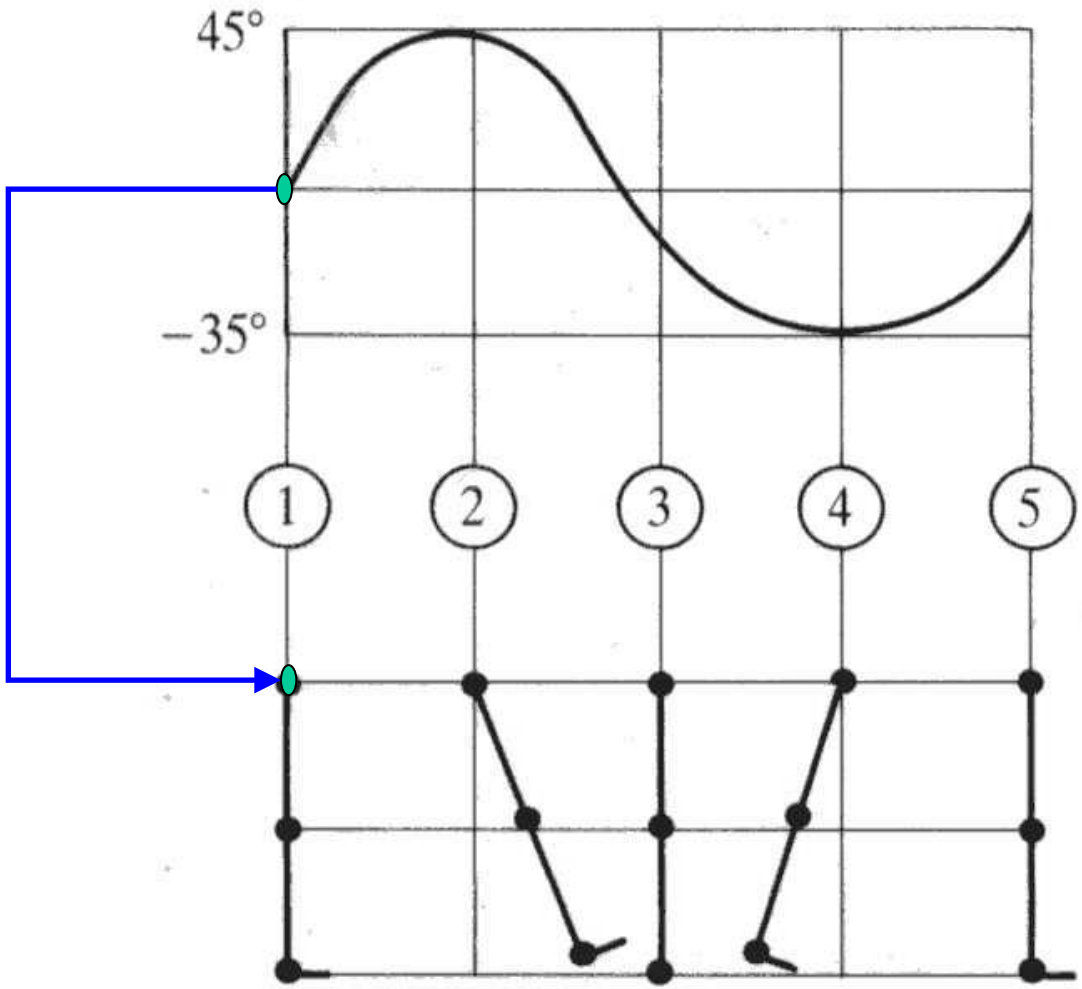
Vorwärtskinematik

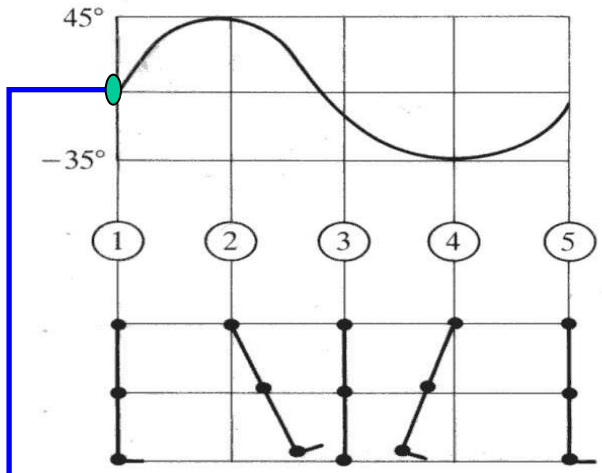
- Animation durch Angabe bzw. Interpolation der Parameter der Gelenke



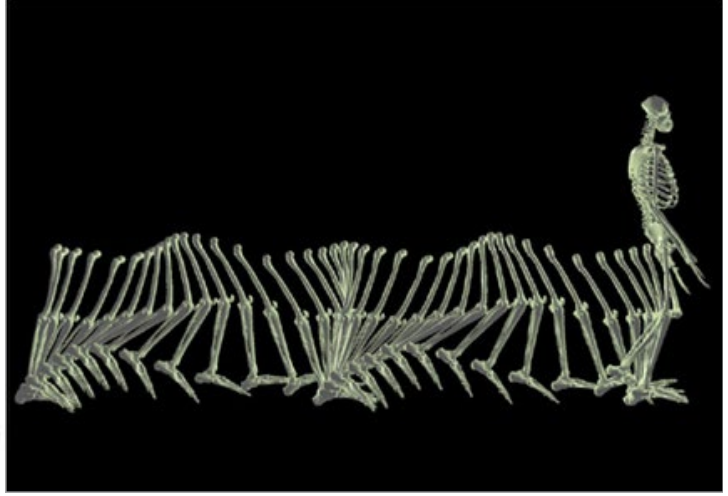
X(t)=f(Theta1(t), Theta2(t))

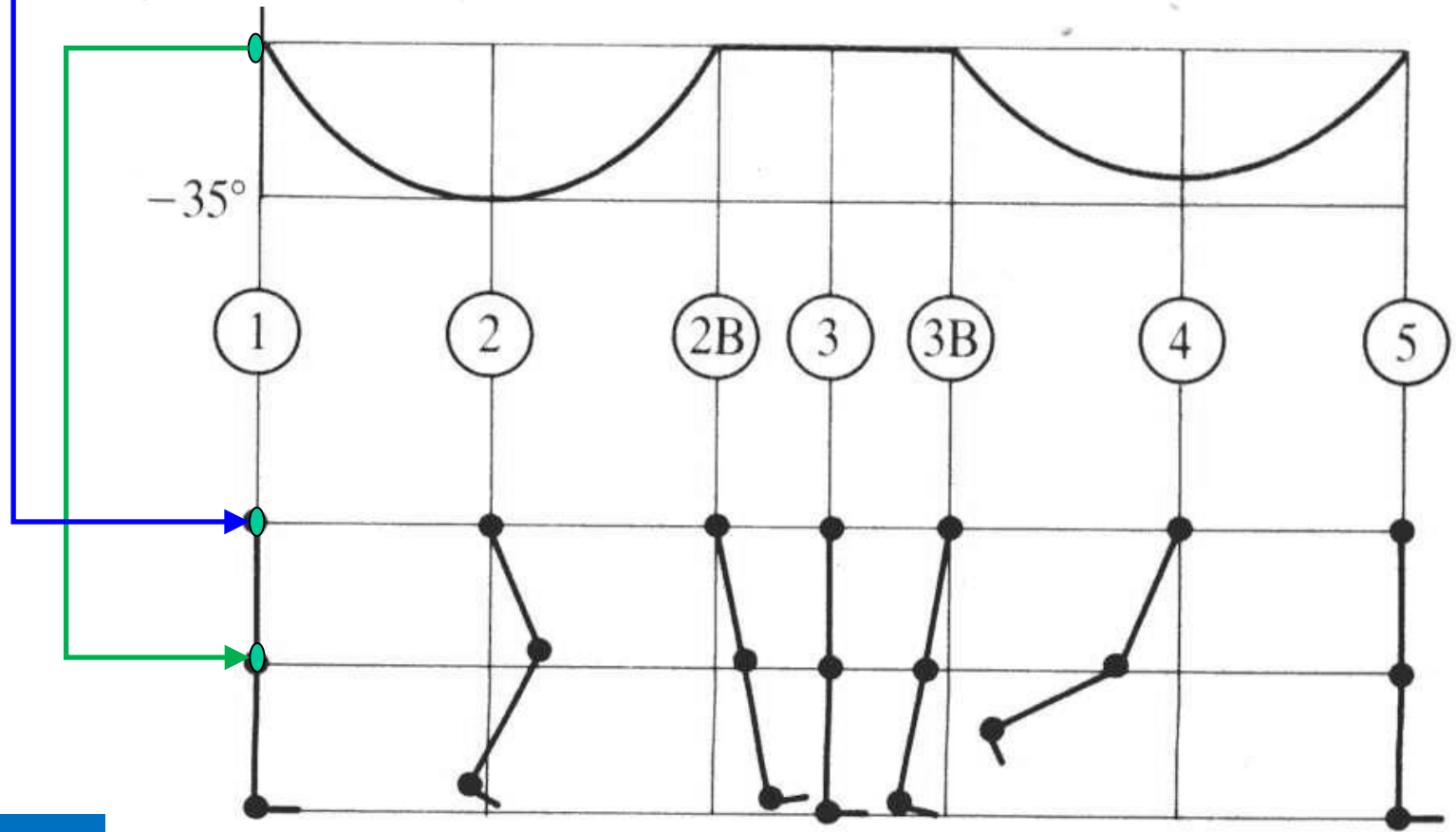
Vorwärtskinematik





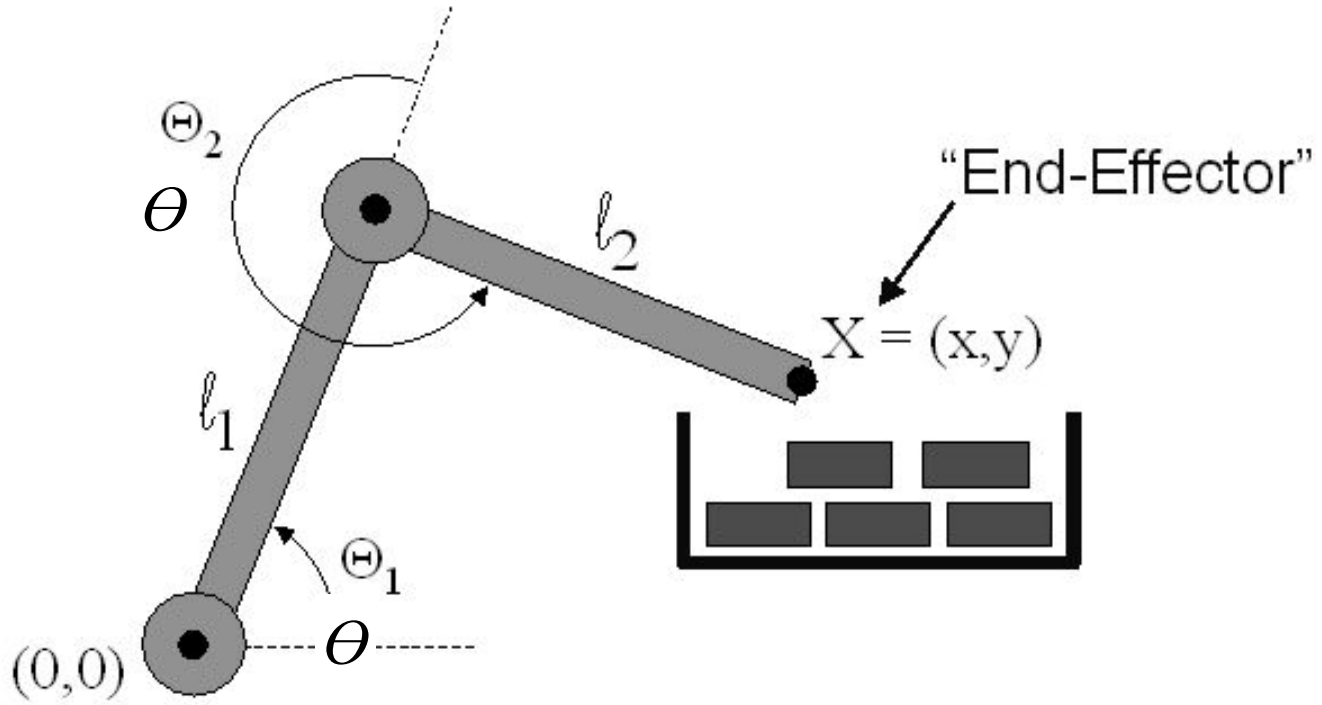
Vorwärts-kinematik



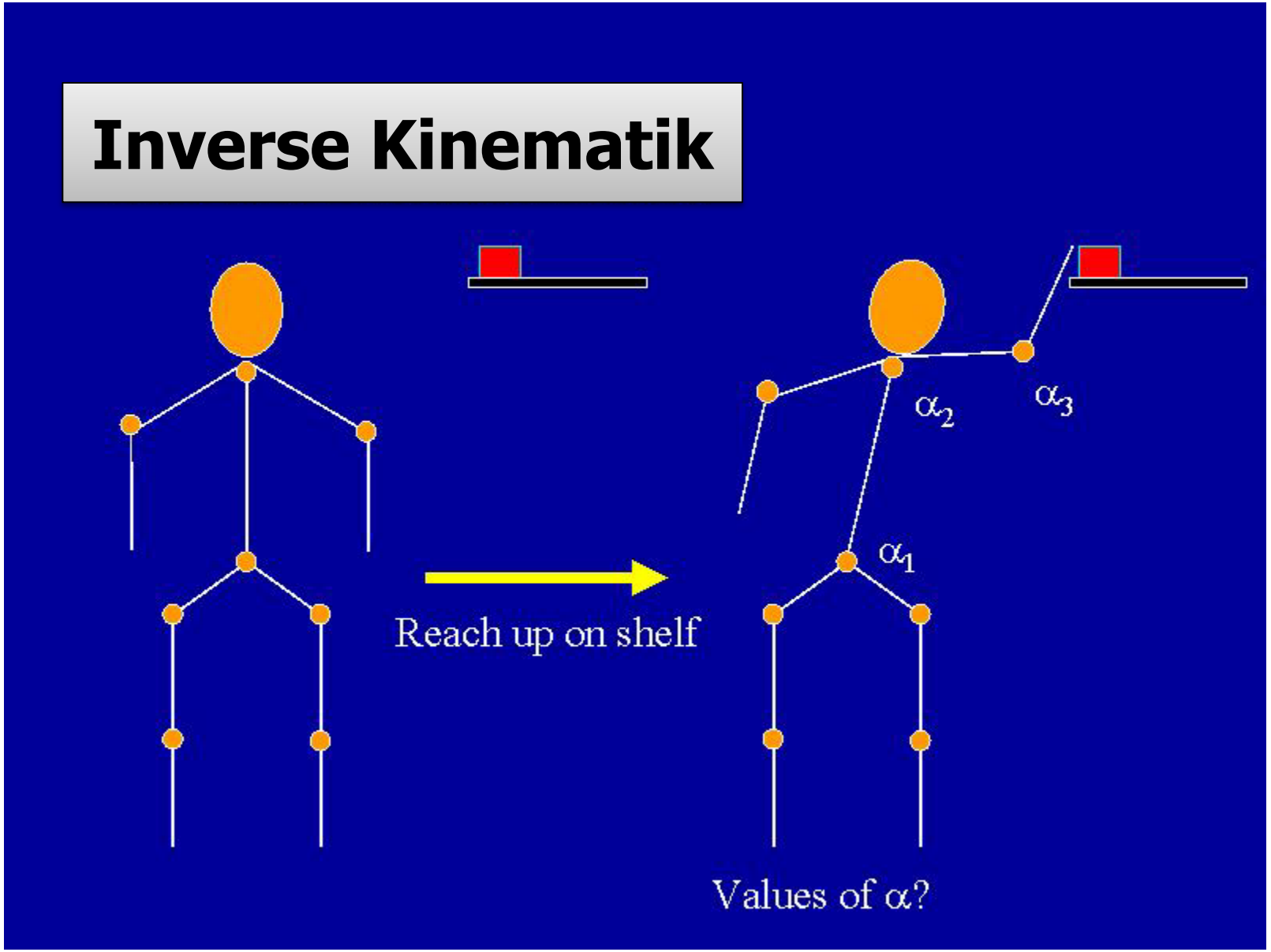


Kinematik

- Was macht man, wenn man statt der Parameter nur die gewünschte Zielposition eines Teilobjekts kennt, z.B. um etwas zu greifen?



Inverse Kinematik



Inverse Kinematik

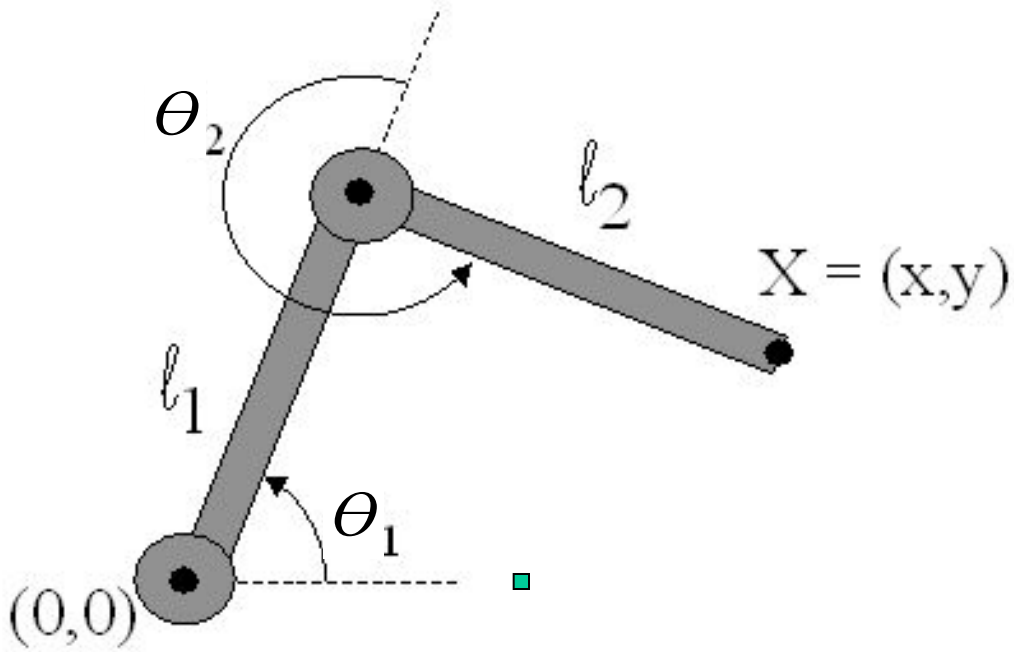
- Animator gibt Position (und Richtung) an, an der die Verbindung (Endeffektor) platziert werden soll.

- Der Computer berechnet die Parameterwerte der einzelnen Gelenke, so dass der Endeffektor an der gewünschten Position landet:

$$\theta_1, \theta_2 = f^{-1}(X)$$

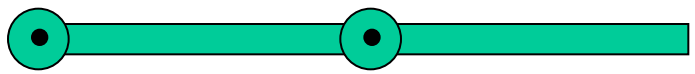
- Anschließend berechnet der Computer die Positionen der Verbindungen unter Verwendung der Parameterwerte.

Inverse Kinematik: Beispiel



Gegeben: Endposition
Gesucht: Winkel

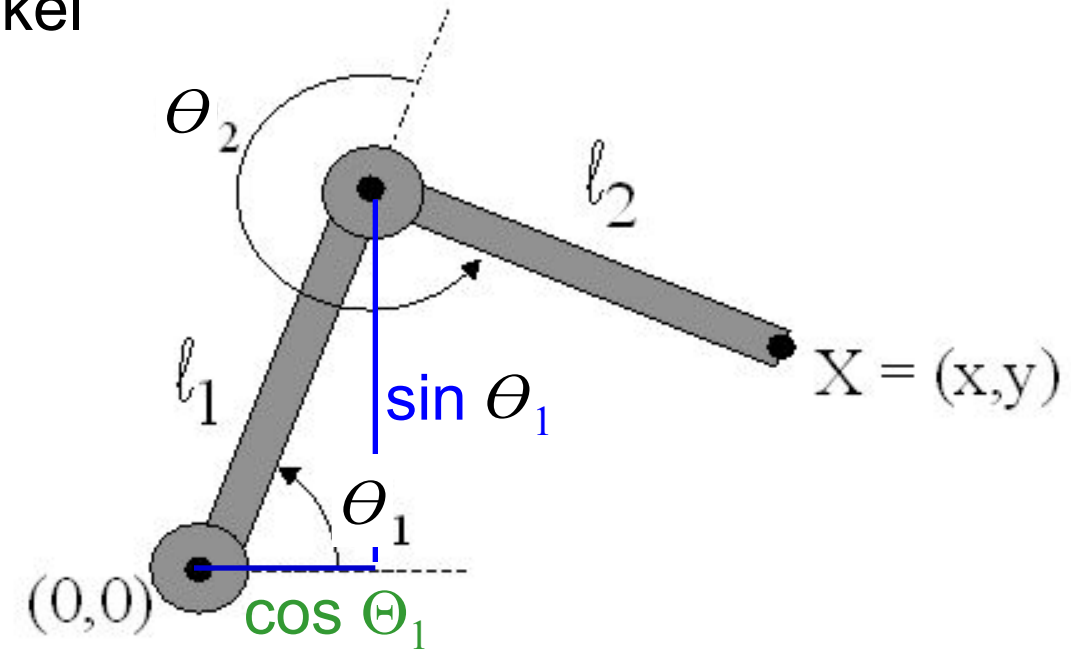
Inverse Kinematik: Beispiel



$\theta_1 = \theta_2 = 0$

Gegeben: Endposition

Gesucht: Winkel



$$X = (l_1 cos\theta_1 + l_2 cos(\theta_1 + \theta_2), l_1 sin\theta_1 + l_2 sin(\theta_1 + \theta_2))$$

Herleitung

$$x = l_1 cos\theta_1 + l_2 cos(\theta_1 + \theta_2)$$

$$y = l_1 sin\theta_1 + l_2 sin(\theta_1 + \theta_2)$$

$$sin^2\alpha + cos^2\alpha = 1$$

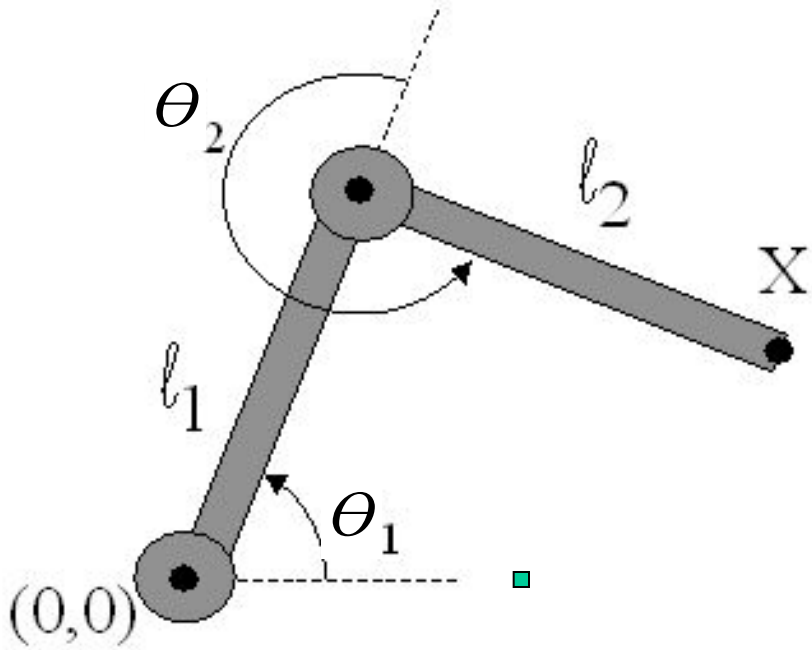
$$\begin{aligned}
x^2 + y^2 = & \quad (l_1 cos\theta_1 + l_2 cos(\theta_1 + \theta_2))^2 + (l_1 sin\theta_1 + l_2 sin(\theta_1 + \theta_2))^2 \\
= & \quad l_1^2 cos^2\theta_1 + l_2^2 cos^2(\theta_1 + \theta_2) + 2 l_1 l_2 cos\theta_1 cos(\theta_1 + \theta_2) \\
+ & \quad l_1^2 sin^2\theta_1 + l_2^2 sin^2(\theta_1 + \theta_2) + 2 l_1 l_2 sin\theta_1 sin(\theta_1 + \theta_2) \\
= & \quad +l_1^2 + l_2^2 + 2 l_1 l_2 (sin\theta_1 sin(\theta_1 + \theta_2) + cos\theta_1 cos(\theta_1 + \theta_2)) \\
= & \quad l_1^2 + l_2^2 + 2 l_1 l_2 cos\theta_2
\end{aligned}$$

$$cos(\alpha - \beta) = cos\alpha cos\beta + sin\alpha sin\beta$$

$$x^2 + y^2 = l_1^2 + l_2^2 + 2 l_1 l_2 cos\theta_2$$

$$cos\theta_2 = \frac{x^2 + y^2 - l_1^2 - l_2^2}{2 l_1 l_2}$$

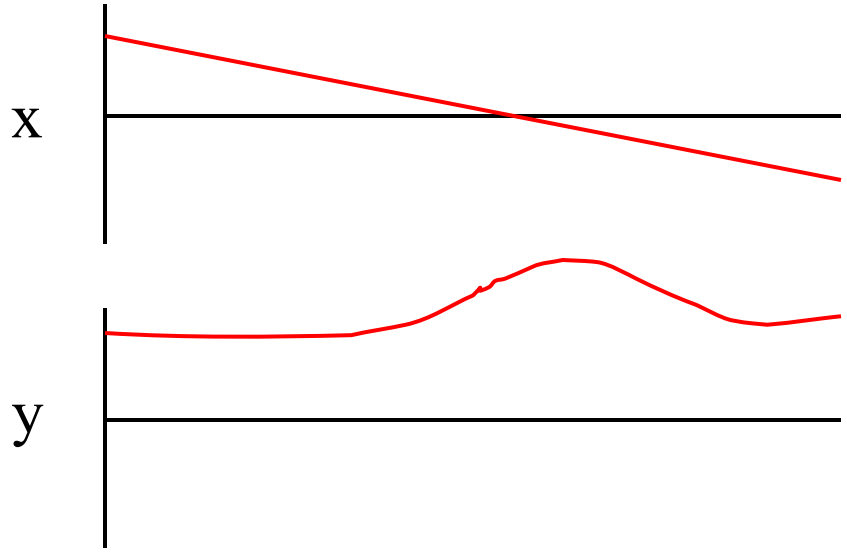
Inverse Kinematik



$$\theta_2 = \cos^{-1}\left(\frac{x^2 + y^2 - l_1^2 - l_2^2}{2 l_1 l_2}\right)$$

Inverse Kinematik

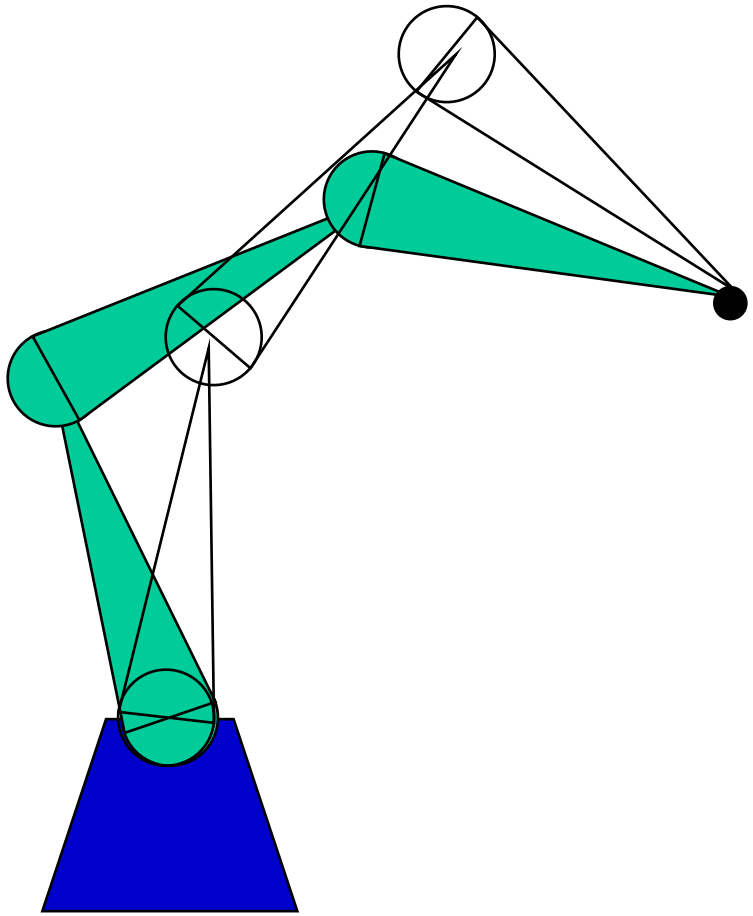
- Animation durch Spezifikation / Interpolation der Position des Endeffektors:



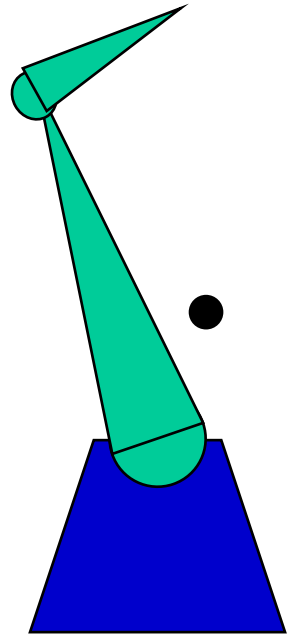
- Alternativ: Animation durch Interpolation aufgrund der Gelenkparameter des Start- und Endframes.

Probleme der Inversen Kinematik

- unendlich viele Lösungen



- keine Lösung



Zusammenfassung

- Vorwärtskinematik
  - Animator steuert über die Gelenkparameter
  - Direkte Steuerung über den Objektzustand
  - Oft müssen viele Parameter gleichzeitig gesteuert werden.
- Inverse Kinematik
  - Animator steuert über die Position/Richtung des Endeffektors
  - Vereinfacht Spezifikation von Bewegungen
    - Weniger Parameter
    - Bessers Gefühl für die Position in der Szene
  - Aufwendige Berechnungen

Chapter 6.3.2

Motion Capture



Übertragung menschlicher Bewegungen auf im Computer generierte 3D-Modelle



Performance Capture = Erfassung von Gestik und Mimik



Lissi und der wilde Kaiser, 2007

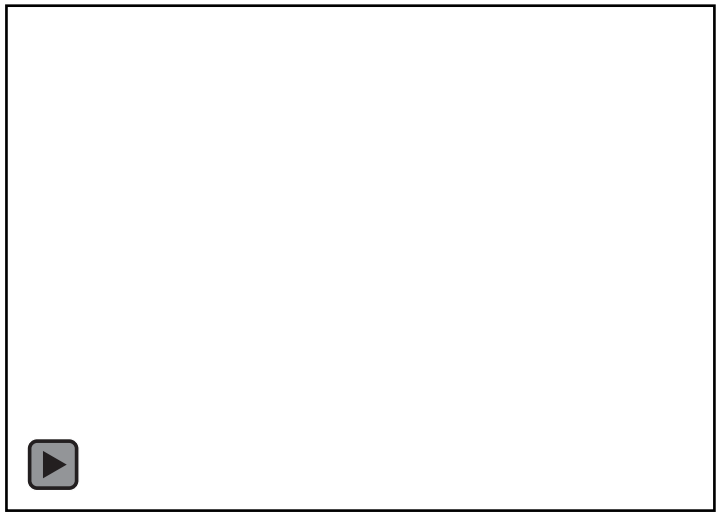
Chapter
6.3.3

Physikalische Simulation



Physikalische Effekte

- Folgende physikalische Effekte beeinflussen u.a., wie sich ein Objekt bewegt:
  - Schwerkraft
  - Turbulenzen, Verwirbelungen (Wind, Wasser)
  - Kontaktkräfte zwischen Objekten
  - Reibung
  - Verbundenheit durch Gelenke



Bewegungsgleichungen

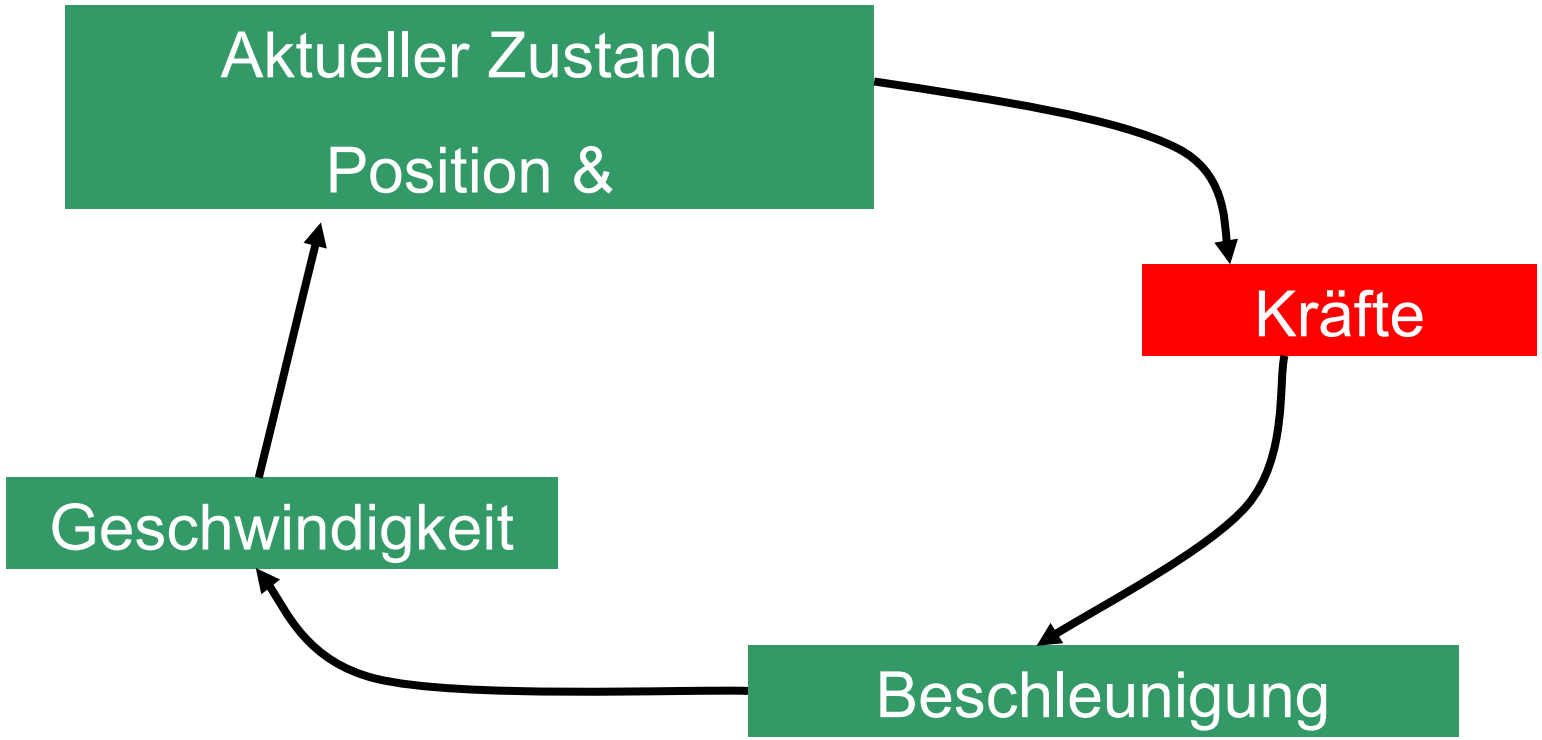
Kraft $F = \dot{p} = ma$

Impuls $p = mv$

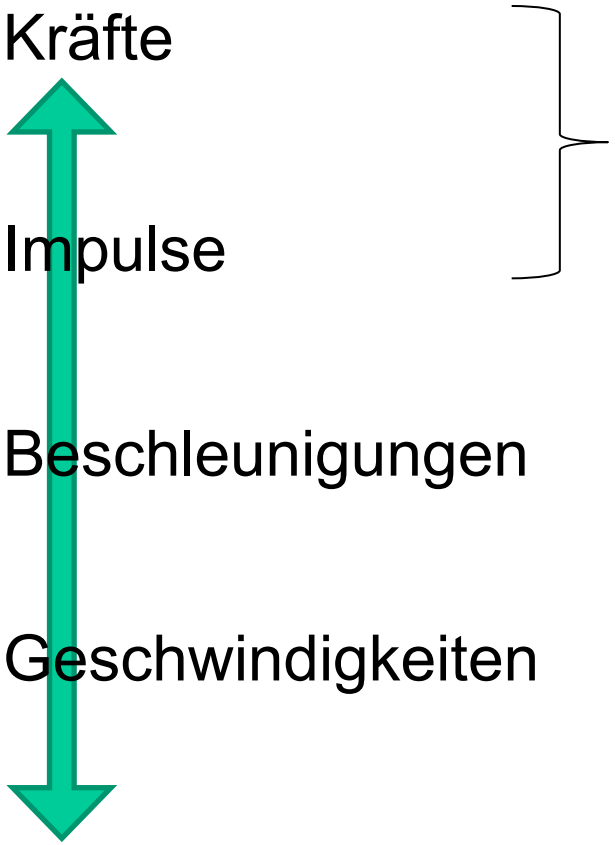
Beschleunigung $a = \dot{v} = \frac{dv}{dt}$

Geschwindigkeit $v = \dot{s} = \frac{ds}{dt}$

Ort (-sänderung)



Physikalische Simulation



Berücksichtigung der Masse

Änderung von Ort, Orientierung, Form etc.

Starrkörper (engl. rigid bodies)

- Starrkörper = **nicht verformbarer** Körper mit Masse, der seine Position und Orientierung ändern kann.
- **Starrkörpersimulation:** Translation und Rotation in Folge von Krafteinwirkung
- **Translation:** Alle Kräfte, die auf einen Körper einwirken, können durch den Summenvektor der einzelnen Kraftvektoren repräsentiert werden, der am Massezentrum auf einen Punkt einwirkt, der die Masse des gesamten Körpers hat.
- **Rotation:** Die Berechnung kann teuer sein, da man den Drehimpuls aller Punkte bestimmen muss, an denen eine Kraft auf den Körper wirkt.

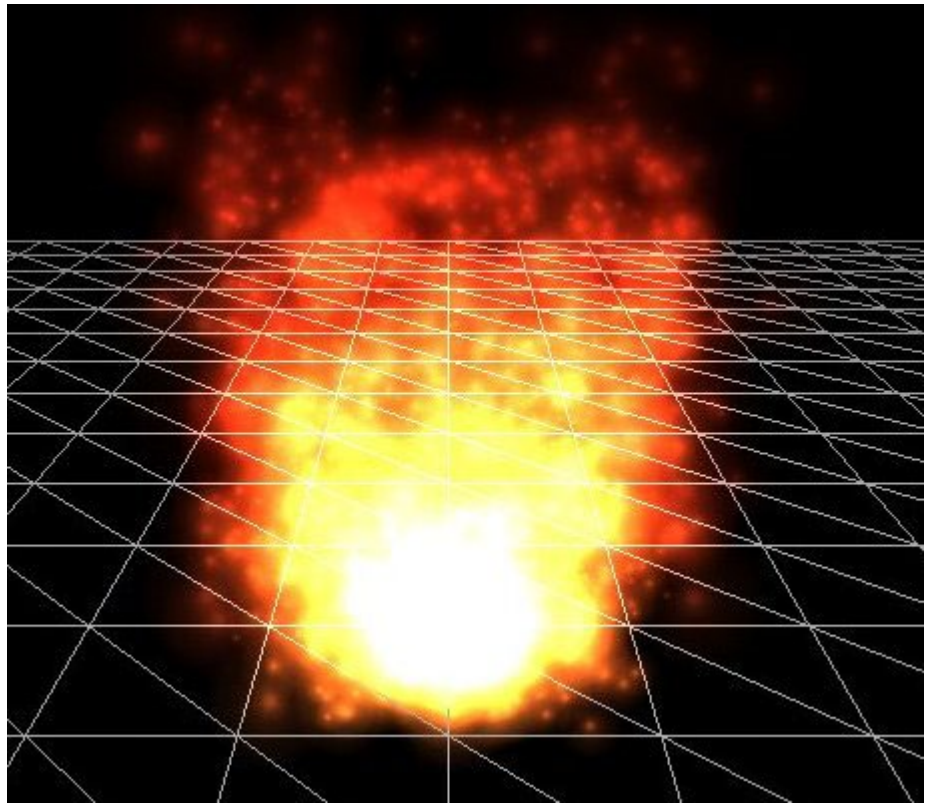
Exkurs

Änderung	Ort s	Orientierung Ω
Kraft	$F = \dot{p} = ma$	$\tau = \dot{L}^{AB} = \vec{r}_{\perp}^{\,AB} \cdot \vec{F}^{B}$
Impuls	$p = mv$	$L^{AB} = \vec{r}_{\perp}^{\,AB} \cdot \vec{p}^{B} = \vec{r}_{\perp}^{\,AB} \cdot m\vec{v}^{B}$
Beschleunigung	$a = \dot{v} = \frac{dv}{dt}$	$\vec{a}^{B} = \dot{\vec{v}}^{B} = \dot{\omega}\vec{r}_{\perp}^{\,AB} = \alpha\vec{r}_{\perp}^{\,AB}$
Geschwindigkeit	$v = \dot{s} = \frac{ds}{dt}$	$\vec{v}^{B} = \omega\vec{r}_{\perp}^{\,AB} = \frac{d\Omega}{dt}\vec{r}_{\perp}^{\,AB}$

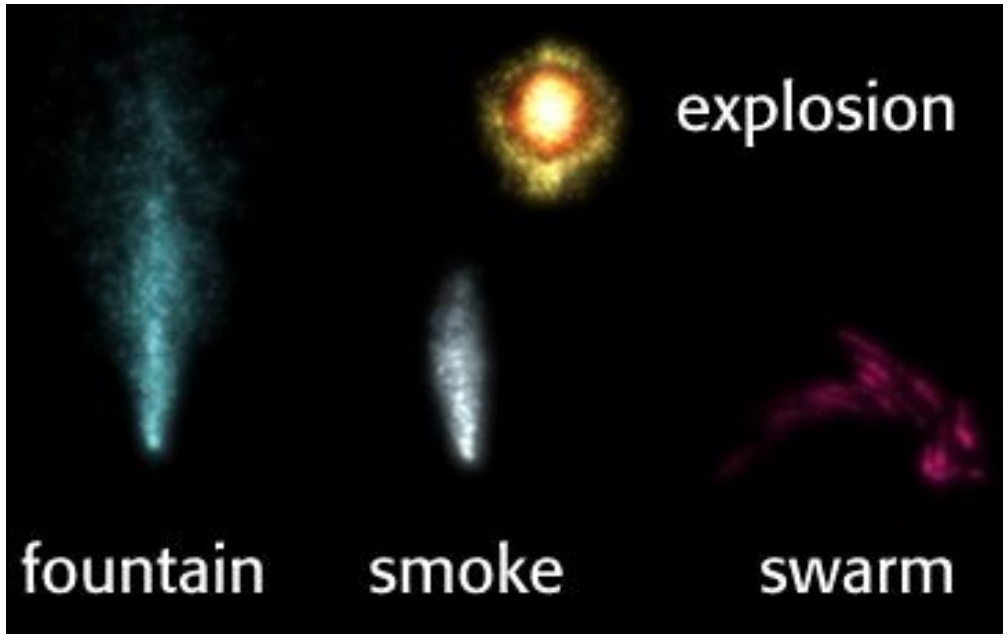
τ: Drehmoment

Chapter
6.3.4

Partikelsysteme



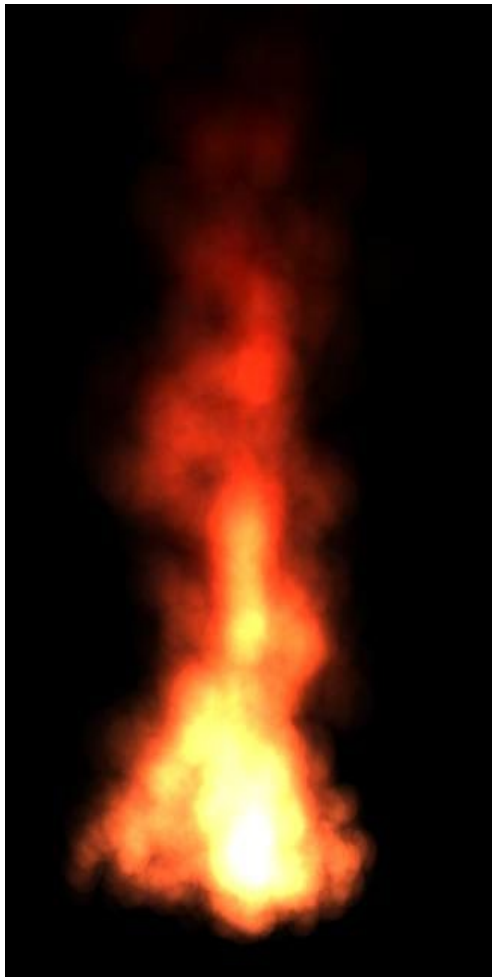
Partikelsysteme



Partikelsysteme erlauben es, amorphe (ungeformt, gestaltlos), dynamische und flüssige Objekte wie Wolken, Feuer, Rauch, Wasser und Explosionen zu modellieren.

Darstellung von Objekten durch Partikel

- Ein Objekt wird als Wolke einfacher Partikel repräsentiert, die das Volumen des Objekts und nicht wie bei Polygonen seine Begrenzung (Umriß) definiert.
- Ein Partikelsystem ist dynamisch. Die Partikel verändern ihre Form und bewegen sich über die Zeit.
- Ein Objekt ist nichtdeterministisch. Seine Form ist nicht vollständig spezifiziert.



Basismodell von Partikelsystemen

1) Im Laufe der Animation/Simulation werden immer wieder neue Partikel **erzeugt**.
2) Jeder neue Partikel hat seine **individuellen Attribute**.
3) Jeder Partikel hat eine vordefinierte **Lebensspanne** und wird danach gelöscht.
4) Die übrigen Partikel werden entsprechend ihrer dynamischen Attribute transformiert und bewegt.
5) Das "Bild" eines Partikels wird oft mit Hilfe spezialisierter Algorithmen gerendert.

Erzeugen von Partikeln

- Partikel werden durch Prozesse erzeugt, die **zufallsgesteuert** sind, d.h. wann und mit welchen Attributen ein Partikel bei seiner "Geburt" versehen ist, hängt vom Zufall ab.

- Eine Möglichkeit die Anzahl erzeugter Partikel zu steuern, besteht darin die **Anzahl neuer Partikel pro Frame** festzulegen

$$n_{neu} = n_{mittel}(t) + (random() \times var(n_{mittel}))$$

- Dieser Ansatz lässt sich verfeinern, in dem man die **Anzahl neuer Partikel pro Bildfläche (screen area)** festlegt:

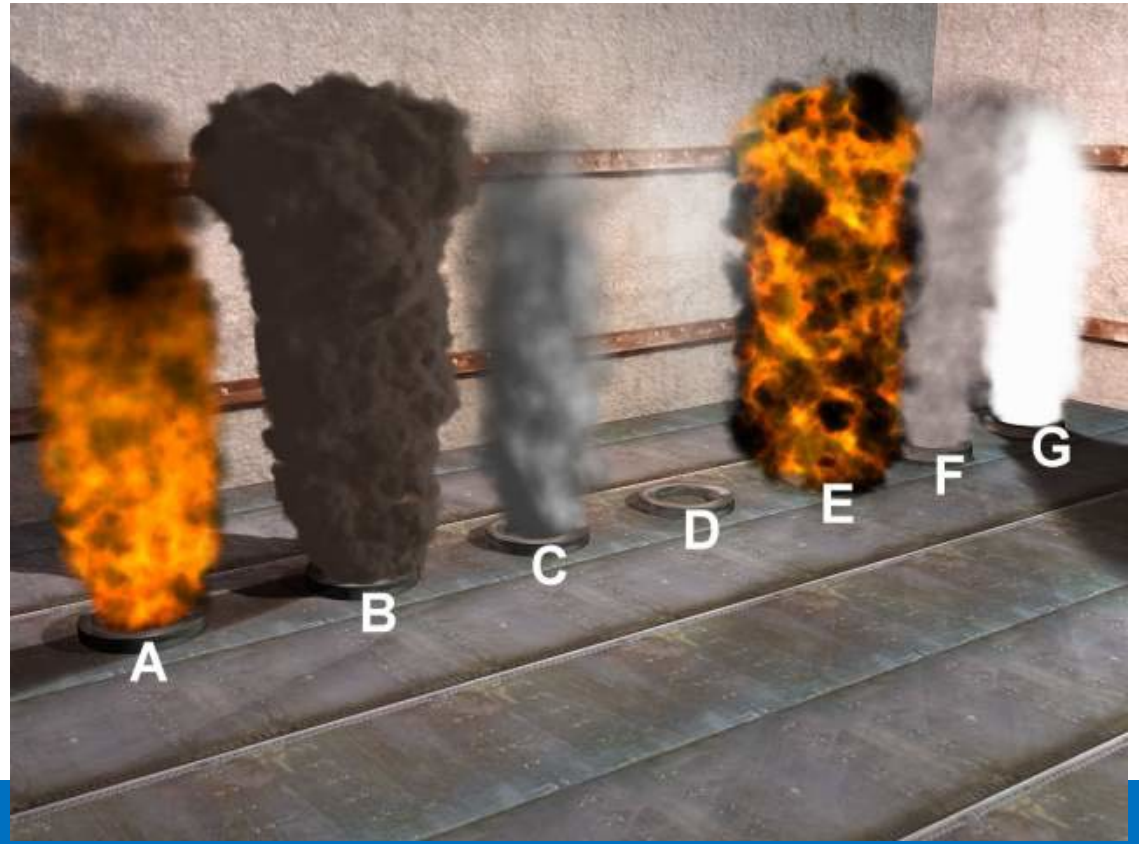
$$n_{neu} = (n_{mittel}(t) + random() \times var(n_{mittel})) \times ScreenArea$$

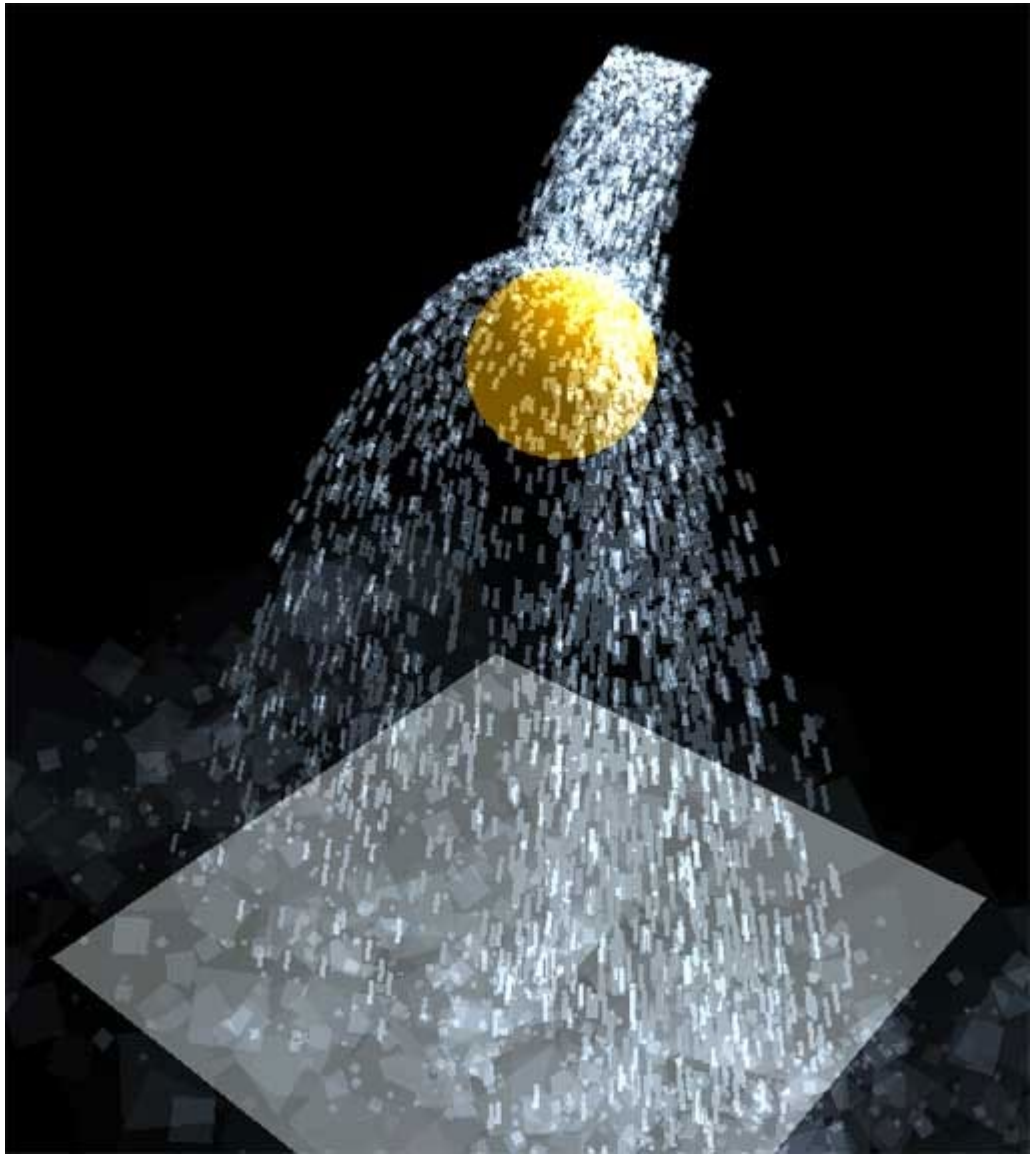
Bei dieser Methode hängt die Anzahl neuer Partikel von der Fläche ab, die das Objekt im Bild einnimmt.

Dynamik von Partikeln

Die Position eines Partikels wird durch Addition seines Geschwindigkeitsvektors zum Positionsvektor berechnet. Dabei können auch Kräfte wie Gravitation oder Wind berücksichtigt werden.

Auch Attribute wie **Farbe**, **Transparenz** und **Größe** können sich über die Zeit ändern. Die Änderung dieser Attribute kann global (für alle Partikel) oder stochastisch für jeden einzelnen Partikel gesteuert werden.

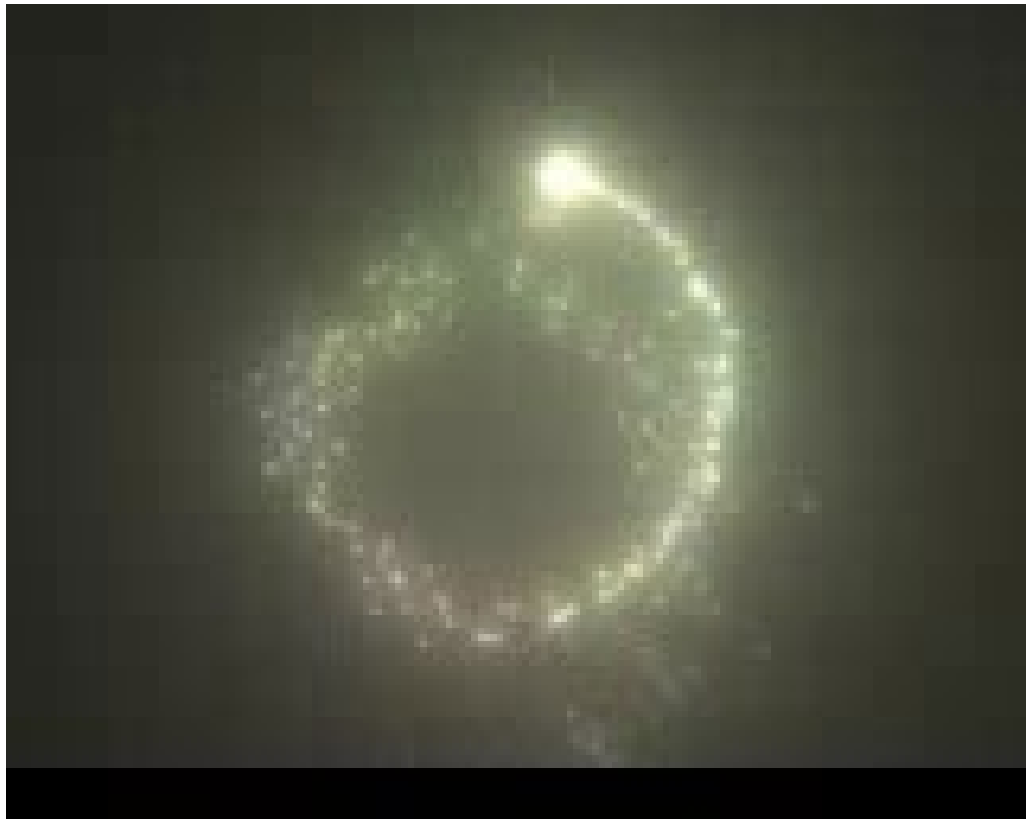




Exkurs

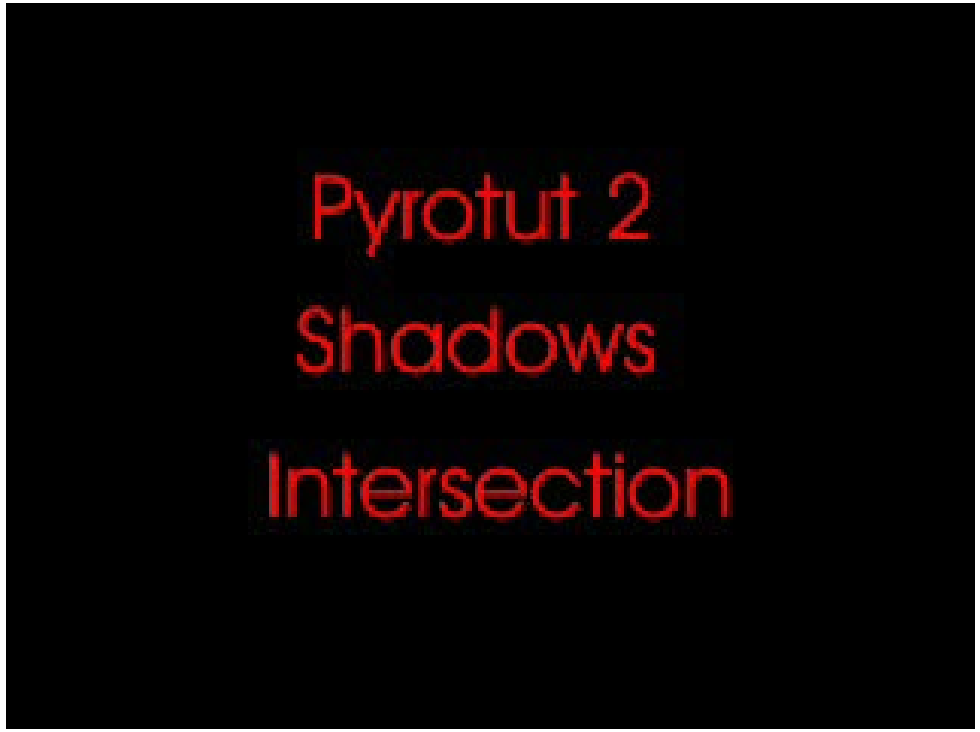
“Sterben” von Partikeln

- Die Lebensspanne eines Partikels wird in Anzahl von Frames spezifiziert.
- Nach jedem Frame wird diese um eines vermindert, ist die Lebensspanne Null, so wird der Partikel gelöscht.
- Insbesondere sollten Partikel gelöscht werden, die nicht mehr zum Bild beitragen, z.B. weil ihre Transparenz zu hoch ist, oder sie aus dem Blickfeld verschwunden sind.



Rendering von Partikeln

- Partikel können dahinterliegende Objekte verdecken, transparent sein und Schatten auf andere Objekte werfen. Die betroffenen Objekte können z.B. Polygone, implizite Flächen oder auch andere Partikel sein.



Exkurs

Star Trek II: The Wrath of Khan

Partikelsysteme wurden erstmals von William Reeves im Jahre 1983 im Film “Star Trek II” eingesetzt.



Startexplosion (1 großes Partikelsystem und 20 kleinere, ca. 25.000 Partikel

Ausbreitung: 200 Partikelsysteme, ca. 75.000 Partikel



Exkurs

Gras als Partikeltrajektorie

- Die gesamte Trajektorie eines Partikels über seine Lebensspanne wird gezeichnet, um ein statisches Bild zu erzeugen.
- Verschiedene Grüntöne werden den Partikeln zugewiesen. Beim Rendern werden die Lichtquellen der Szene beachtet.
- Aus jedem Partikel wird so ein Grashalm.



white.sand von Alvy Ray Smith
(ehemaliger Mitarbeiter von Lucasfilm)

Exkurs

Attribute von Partikeln in Maya

- Anfangsposition
- Anfangsgeschwindigkeit
- Anfangsgröße
  - InitialSize = MeanSize + Rand() X VarSize
- Anfängliche Farbe
- Anfängliche Transparenz
- Form (Shape)
- Lebensspanne

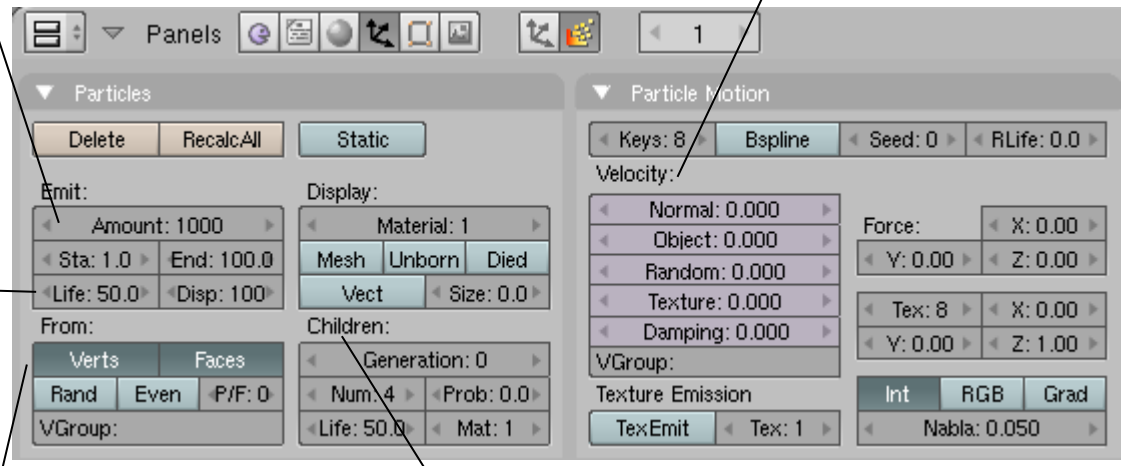


Alias|Wavefront's Maya

Attribute von Partikeln in Blender

verschiedene Möglichkeiten die Geschwindigkeit eines Partikels zu spezifizieren

Anzahl (Summe über alle Generationen)



Lebensspanne

Position/Ort an der die Partikel erzeugt werden sollen

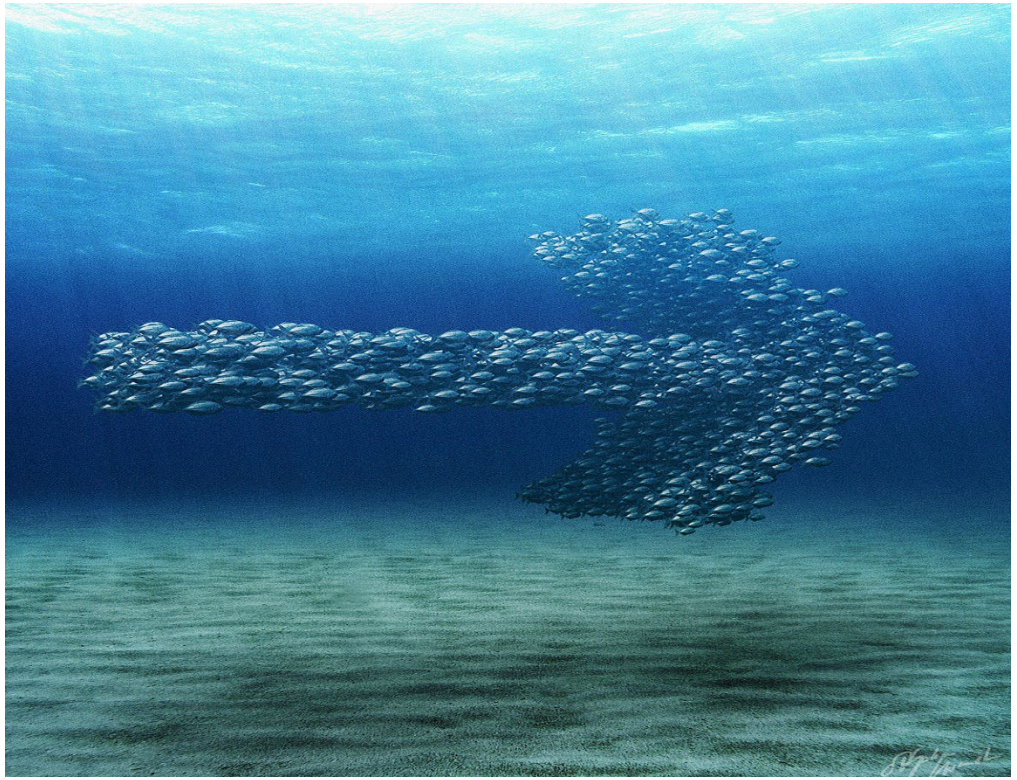
„Sterbende“ Partikel können neue erzeugen

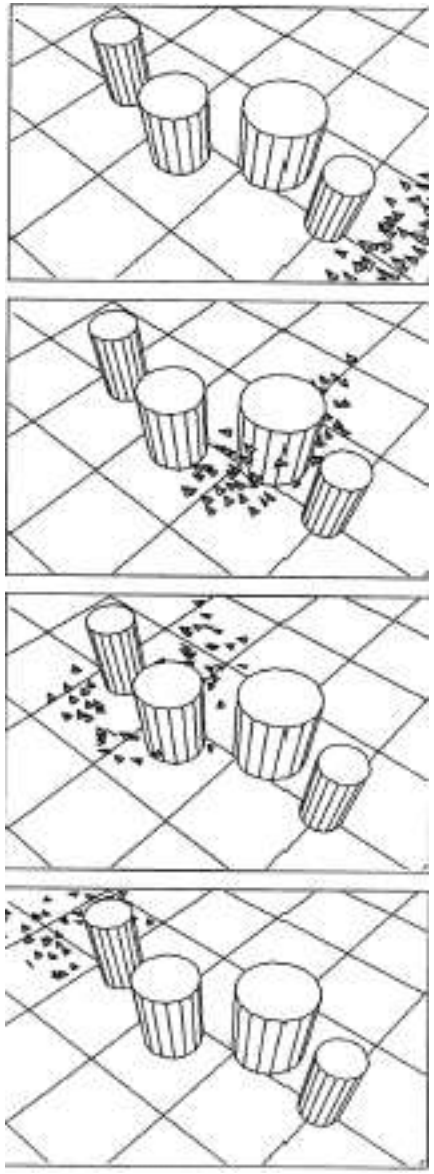
Partikel in Three.js

- http://threejs.org/examples/#webgl_points_dynamic
- http://threejs.org/examples/#webgl_gpu_particle_system
- http://stemkoski.github.io/Three.js/Particle-Engine.html
- http://jeromeetienne.github.io/tquery/www/live/editor/#U/../../../plugins/particles/examples/smokepuff.html

Chapter
6.3.5

Schwärme





Schwärme

- Erste komplett animierte Filmsequenz eines komplexen Vogelschwarms (Reynolds,1987)
- Jedes Schwarmmitglied („Akteur“) befolgt drei einfache Regeln:

→ Kollisionsvermeidung

– Akteure vermeiden selbstständig Kollisionen

→ Geschwindigkeitsanpassung

– Akteure passen Geschwindigkeit den Nachbarn an

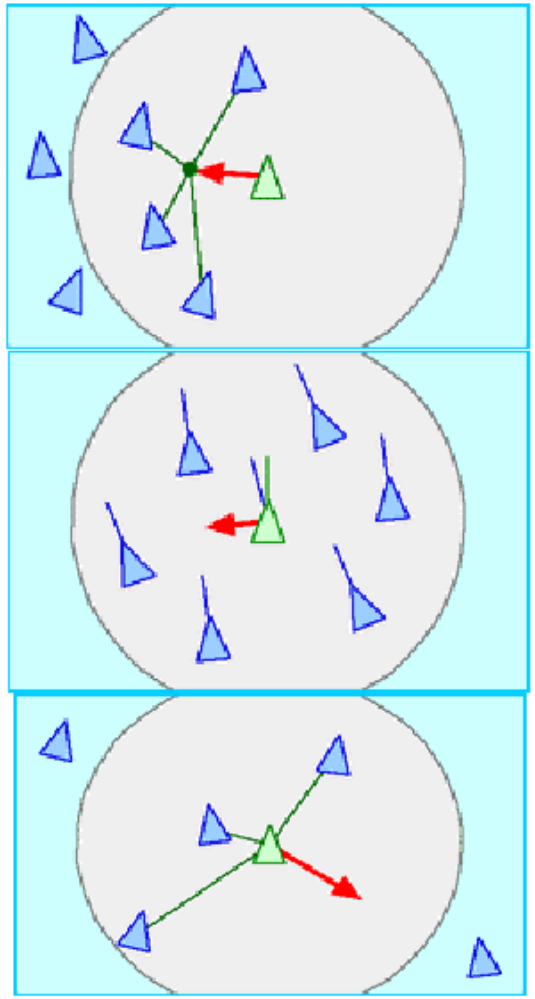
→ Schwarmzentrierung

– Akteure versuchen zusammenzubleiben

[1] Reynolds, Craig W. (1987) Flocks, Herds, and Schools: A Distributed Behavioral Model, in Computer Graphics, 21(4) (SIGGRAPH '87 Conference Proceedings) pages 25-34.

Schwärme

Für jeden Akteur werden 4 Nachbarn gespeichert.
Aus diesen Nachbarn wird das Steuerverhalten berechnet:



„Cohesion“: Bewege dich auf die durchschnittliche Position zu.

„Alignment“: Steuere in die durchschnittliche Flugrichtung

„Separation“: Vermeide zu dichte Ansammlung von Schwarmmitgliedern.

Vergleich

Partikelsysteme:

- begrenzte Lebenszeit der Partikel
- Gesamteindruck ist maßgebend
- Partikel sind wenig komplex
- Partikel folgen passiv dem Bewegungsablauf

Schwärme:

- Lebenszeit nicht begrenzt, solange wie der Schwarm
- dynamischer Eindruck ist maßgebend
- komplexe Akteure
- Akteure greifen aktiv in das Geschehen ein

THE END